NEPANTLA

Guaranteed Non-Inferior Execution for a Spanish-Language
LLM Operator System under Arbitrary Backend-Model Capability

System under study: **Tlamatini** — 86 workflow agents, 44 direct tools,
a Multi-Turn operator loop and a visual flow designer

Creator of the system: **Angela López Mendoza** (XAIHT)

Analysis: Claude Opus 5 — forensic multi-agent study of the codebase
at commit `3e6d514f` (v1.47.0)

27 July 2026

Abstract

We address a problem that arises whenever a locally-deployed AI *operator* system — one that selects tools, extracts byte-exact arguments and mutates real machine state — must serve a user in a language other than the one it was built in, while the user retains the freedom to bind **any** backend model, including models with weak or absent competence in that language.

The naive formulations both fail. Translating the user's request into English before processing injects a *conjunctive* failure mode: an operator task fails completely if any single file path, flag or identifier is damaged, so fidelity decays as p^k in the number of literals and reaches ~11% absolute failure at realistic density even at a generous per-literal preservation rate of $p = 0.98$. Conversely, simply passing Spanish through and hoping the model copes makes correctness a *function of an unknown, user-chosen model* — which is precisely the dependency a product cannot accept.

We propose **NEPANTLA** — *Non-inferior Execution via Progressive Augmentation, Native-Token Locking and Arbitration* — an algorithm that removes the dependency entirely. Its central move is to stop treating cross-lingual operation as a language problem and treat it as a **verification** problem. Three results make this possible.

First, in an operator system the observable outcome factorizes into an *execution outcome* (the state of the machine) and a *presentation outcome* (the text the user reads), and these are conditionally independent given the emitted action trace (**Proposition 1**). Execution correctness can therefore be guaranteed independently of the answer language — *provided* the action vocabulary itself is language-invariant.

Second, every correctness-bearing literal can be extracted from the raw utterance before any transformation and re-verified byte-exactly in the emitted action, converting the p^k decay to $p^k = 1$ (**Proposition 2**). This yields **argument provenance** — the requirement that every literal in an emitted action be derivable from the user's own words, a prior tool result, or a declared default — which is a *language-independent correctness certificate* and the sharpest cross-lingual signal available.

Third, and decisively: if failures are *detected* rather than accepted, an escalation ladder whose terminal rung is operationally identical to the English baseline **cannot perform worse than that baseline** (**Theorem 1, Ladder Dominance**). The guarantee therefore reduces entirely to the strength of a verifier — and in an operator system the verifier can be made strong, because actions are structured objects rather than prose.

Two corollaries follow that materially change the engineering. **Probe Safety** (**Corollary 1**): a model-capability probe influences only the *expected number of rungs*, never the terminal correctness, so probe miscalibration costs latency and never correctness. **Naming Necessity** (**Corollary 2**): Proposition 1 holds only while the action vocabulary is language-invariant — therefore keeping every agent name, tool name, flag, configuration key, protocol sentinel and internal identifier **byte-identical in English across all locales is not a stylistic preference but a precondition of the guarantee.** Translate `Emailer` and the proof fails.

We also report a measurement that reframes the engineering priority. Executing the system's real capability scorer over its real 64-agent registry with twelve matched prompt pairs yields **English top-1 tool selection 10/12 and Spanish 0/12**, with the semantically correct agent scoring *exactly zero* in 7 of 12 Spanish cases — not because the models are weak in Spanish (for Spanish the model-side gap is 1–4 points on reasoning and approximately zero on tool selection) but because the system's own deterministic control plane is monolingual at the tokenizer level. We give three language-neutralization operators that repair this with a proven identity guarantee on ASCII input, so English behaviour is provably unchanged.

Finally we specify how the claim is to be *falsified* rather than asserted: a pre-registered non-inferiority protocol ($\delta = 0.05$ on programmatic task success, Tango score intervals on paired proportions, 250 paired items, deliberately-degraded positive controls establishing assay sensitivity).

The scope of translation is stated once and held throughout: ***the graphical interface becomes fully Spanish — menus, verbs, instructions, messages, wording — and everything the machine consumes stays English.***

Status: research paper. No Tlamatini source code was modified in its production. Every codebase claim was obtained by executing the real functions against the real registries; every external citation was independently re-fetched and verified.

Contents

The name. Nepantla *is Nahuatl for* "in the middle" *— the space between two worlds, neither one nor the other, where something new is made. Tlamatini (*"the one who knows"*) is named in the same language. A Spanish-speaking operator commanding an English-built machine lives in nepantla, and this paper is about making that middle place* ***safe*** *rather than lossy.*

1 Introduction

1.1 The requirement

Tlamatini must become fully usable by a Spanish-speaking operator. Concretely, three things are required simultaneously, and the third is what makes the problem hard.

1. **The interface must be Spanish.** Every menu, button, dialog, tooltip, placeholder, status message, error, instruction and explanatory sentence the user reads is rendered in Spanish. Not partially. Not "the important parts".
2. **The interaction must be Spanish.** The user writes in Spanish, thinks in Spanish, and receives answers in Spanish.
3. **The result must be at least as correct as the English build — for any backend model the user chooses to bind.**

Requirement 3 is not a quality aspiration; it is a hard constraint, and it is unusual. Tlamatini lets its operator select the model: a local Ollama build, an Anthropic endpoint, a cloud alias such as `glm-5.2:cloud`, a small quantized model on a modest GPU, a vision model. Some of those models are excellent in Spanish. Some are mediocre. Some are effectively English-only. **The product cannot know in advance, and must not require the user to know.**

A system that says *"Spanish works, as long as you pick a good model"* has not solved the problem. It has moved it onto the user, who by construction is the person least equipped to evaluate a model's Spanish competence — and who will discover the deficiency only through a silently wrong file deletion.

1.2 Why this is not a translation problem

The instinct is to reach for machine translation, and the instinct is wrong here for a reason specific to this class of system.

Tlamatini is not a chat assistant that produces prose. It is an **operator**. Its output is not a paragraph; it is a *sequence of actions on real state*: create this file at this exact absolute path, flash this firmware to this board over this port at this baud rate, delete these files matching this glob, send this message to this recipient. The correctness of the answer is not "does the sentence read well" but **"did the machine do the right thing"**.

That distinction has a mathematical consequence. Prose quality is *additive* and degrades gracefully — a translation that damages one word in twenty loses roughly five percent of its value. An action is *conjunctive* and degrades catastrophically — a tool call that damages one path in twenty **fails entirely**. There is no partial credit at the filesystem. §4 makes this precise.

So the design question is not *"how do we translate well?"* but:

Where in the pipeline is it safe for a lossy transformation to sit?

The answer, developed in §3.4, is sharp: a lossy transformation may sit **downstream of a verified decision** and may never sit **upstream of one**. This single placement rule dissolves most of the apparent tension in the problem, because it makes translation a *presentation* technique rather than a *comprehension* technique.

1.3 What "at least as correct" must mean

Informal parity claims are unfalsifiable. We fix the meaning now and hold it throughout.

Let $\mathcal{T}$ be a task drawn from the operational distribution, rendered as an English utterance u_{en} and a semantically equivalent Spanish utterance u_{es} carrying **byte-identical literals**. Let $\text{succ}(\cdot)$ be a machine-checkable success predicate evaluated against the state of the machine after execution — file present at the exact path with the exact hash, process exit code, log regex, database row, generated flow compiles.

Write $\theta_{EN} = \Pr[\text{succ}(B(u_{en}))]$ for the English baseline pipeline B, and $\theta_{ES} = \Pr[\text{succ}(A(u_{es}))]$ for the Spanish pipeline A.

The requirement is $\theta_{ES} \geq \theta_{EN}$, established two ways:

- **By construction** (§6.7): an architectural argument that A dominates B, from which the inequality follows without measurement.
- **By experiment** (§8): a pre-registered non-inferiority test at margin δ, because an architectural argument that has never been measured is a hypothesis wearing a proof's clothing.

Both are required. The first tells us the design cannot be *structurally* inferior; the second tells us the implementation is not.

1.4 Contributions

1. **A translation boundary** (§2) that partitions every string in the system into *presentation* and *machine* classes, with an explicit, exhaustive, non-negotiable invariant on the machine class — including the naming rule that agent identities such as `Emailer`, `Asker`, `Apirer` and `STM32er` remain byte-identical in every locale, forever.
2. **A formal framework** (§3) with two propositions — Execution/Presentation Separation and Literal Anchoring — and the **Translation Placement Principle**.
3. **A measured characterization of substrate monolingualism** (§5): a real 700-file operator codebase whose deterministic control plane collapses under Spanish input, with executed measurements rather than inspection, and three repair operators carrying an identity guarantee on English input.
4. **NEPANTLA** (§6): the algorithm. Literal freezing, a six-check language-independent verifier including **argument provenance**, a five-rung escalation ladder, an independent presentation renderer, full pseudocode, the **Ladder Dominance** theorem and its two corollaries, complexity analysis and failure-mode analysis.

5. **A treatment of the not-Spanish-capable model** (§7) that makes model competence an *optimization* rather than a *dependency*, with three worked execution traces.
6. **A falsifiable evaluation protocol** (§8) with assay sensitivity, so the guarantee can be disproved.

1.5 Method

Every claim about the codebase was produced by fourteen parallel analysis agents reading the repository directly (684 tool invocations), each required to attach `file:line` evidence it had actually observed, and every empirical result reported in §5 was obtained by *executing* the real functions against the real registries rather than by reading them.

Every external scientific claim was then subjected to an independent adversarial verification pass which re-fetched each cited work to confirm title, authors, year and identifier, and re-checked each numeric value against the source table. That pass identified one misattributed reference, which was removed. Two further adversarial agents attempted to refute the architecture; their surviving objections were incorporated into the design rather than rebutted, and are visible in the final shape of §6 and §7 — the read-only guard, the per-path routing, the provenance check and the conversation-level hysteresis all exist because an attack landed.

2 The Translation Boundary

Before any algorithm, the scope must be exact. This section is the contract.

2.1 Two channels, one rule

Every string in the system belongs to exactly one of two channels.

Table 1: The two channels. Every string in the system belongs to exactly one of them.

	Presentation channel	**Machine channel**
Definition	Read by a human, never read back by a program	Consumed, compared, routed on, persisted, or parsed by a program
Audience	The Spanish-speaking operator	The interpreter, the filesystem, the parser, the model's tool interface
Language	**Spanish** (fully)	**English, byte-identical, forever**
On error	Cosmetic — a clumsy sentence	Catastrophic — wrong tool, wrong path, silent corruption

The rule is one sentence:

Translate what the user reads. Never translate what the machine reads.

2.2 What becomes Spanish

The entire visible surface. This is not a subset; it is everything a human eye lands on.

- **Navigation and chrome** — the navbar menus and every dropdown item, dialog titles, section legends, tab labels, panel headings.
- **Controls** — every button caption, checkbox and toggle label, radio label, the toolbar, the Send/Cancel affordance, context menus.
- **Input affordances** — placeholders, tooltips, `aria-label` accessibility text, helper text, character counters.
- **Generic verbs and nouns of the interface** — *Guardar, Cancelar, Continuar, Cerrar, Eliminar, Ejecutar, Detener, Validar, Iniciar, Pausar, Reanudar, Buscar, Importar, Exportar, Actualizar, Reintentar.*
- **Instructional prose** — wizard steps, the setup guidance, the "how to use this" copy, empty-state explanations, confirmation questions.
- **Feedback** — status lines, progress messages, success and failure notices, validation warnings, permission prompts, the interrupted-execution banner.
- **Report chrome** — the Exec Report's title, its column headers (*Comando*, *Estado*), its verdict words (*ÉXITO*, *FALLO*), the denial banner's labels.
- **Catalog content** — the prompt catalog's category names and card descriptions.
- **Agent descriptions** — the tooltip and description text explaining what each agent *does*.
- **The model's answer prose** — the sentences Tlamatini writes back.
- **Documentation and onboarding** intended for the operator.

2.3 What NEVER becomes Spanish

This list is exhaustive by *class*, and it is absolute. A violation here is not a cosmetic bug; §6.9 proves it invalidates the correctness guarantee.

2.4 The Naming Invariance Rule

Stated as an operational rule an engineer can apply without judgement:

Principle 1 (RULE N)**.** An agent's display name is an **identifier**, not a label. It is byte-identical in every locale, in every surface, forever — including its exact capitalization, its hyphens and its spaces. What is translated is the agent's *description*, never its *name*.

Applied concretely, in the Spanish build:

The pattern is uniform: **Spanish sentence, English name embedded verbatim.** This is also how Spanish-speaking engineers already write and speak about tooling — *"corre el linter"*, *"revisa el commit"*, *"el flag `-noreload`"* — so the result reads naturally rather than as an untranslated remnant.

2.5 Why the boundary is a precondition, not a compromise

It would be easy to read §2.3 as a limitation accepted for expediency. It is the opposite. The naming invariant is what *creates* the guarantee.

Table 2: The machine channel, enumerated by class. Every entry stays byte-identical in English in every locale, forever.

Class	Examples (illustrative, not exhaustive)
Agent display names	`Emailer`, `Asker`, `Apirer`, `Executer`, `Pythonxer`, `STM32er`, `Kyber-KeyGen`, `File-Creator`, `De-Compresser`, `Monitor-Log`, `Node Manager`, `TeleTlamatini`, `RecMailer`, `FlowCreator`
Tool names	`chat_agent_send_email`, `chat_agent_apirer`, `execute_command`, `acp_spawn`, `invoke_skill`, `ext__<server>__<tool>`
Agent directory / pool names	`agents/emailer/`, `emailer_1`, `apirer_2`
Configuration keys	`unified_agent_model`, `django_port`, `binary_context_detection`, `pio_executable`
Configuration field names inside `config.yaml`	`target_agents`, `source_agents`, `pattern_a`, `outcome_word`, `capture_mode`
Protocol sentinels	`END-RESPONSE`, `BEGIN-CODE«<...»>` / `END-CODE`, `INI_SECTION_<TYPE>«<` / `»>END_SECTION_<TYPE>`, `TLM_VERDICT::PASS_OK`, `VERDICT: REQUEST_CHANGES`
Machine verdict vocabularies	`APPROVE`, `REQUEST_CHANGES`, `COMMENT`, `PASS_OK`, `FAIL_NO_MOTION`, `UNCLEAR`, `completed`, `failed`, `stopped`
Source code	Every identifier, function name, class name, module name, comment, docstring
Internal variable nomenclature	`_score_capability`, `exec_report_entries`, `answer_language`, `cancel_run_epoch`
CLI flags and switches	`-noreload`, `-self-modify`, `-sT`, `-collect-all`
File and directory names	`prompt.pmt`, `tlamatini.log`, `Temp`, `Templates`, `config.json`, `.flw`
CSS classes, element ids, `data-*` attribute values	`.canvas-item.stm32er-agent`, `#prompts-catalog`, `data-content="Emailer"`
Log-line prefixes	`-- [BINARY-GUARD]`, `-- [I18N-GUARD]`, `AGENT STARTED`
Environment variables	`TLAMATINI_TEMP`, `AGENT_REANIMATED`, `PDCP_API_KEY`
`.flw` schema keys	`agentName`, `configData`, `connections`, `schemaVersion`
Brand and authorship	`Tlamatini`, `XAIHT`, `Angela López Mendoza`, `ACPX`, `Multi-Turn`, `Exec report`, `Ask Execs`, `System-Metrics`, `Files-Search`
User-supplied literals	Absolute paths, filenames, hostnames, ports, board identifiers, git refs, regular expressions, glob patterns

Table 3: Per-surface rendering in the Spanish build. The pattern is uniform: Spanish sentence, English name embedded verbatim.

Surface	**Spanish build renders**
Sidebar item	**Emailer**
Tooltip on that item	*"Envía correos electrónicos por SMTP cuando se detecta un patrón en el registro."*
Canvas node label	**Emailer**
Right-click description dialog title	**Emailer**
Exec Report caption	*Lista de operaciones de* **Emailer**
Exec Report column headers	*Comando · Estado*
Exec Report verdict cell	*ÉXITO / FALLO*
Permission prompt	*"¿Permitir que se ejecute* **Apirer** *?"*
Answer prose	*"Ya lancé* **Asker** *y está esperando tu elección."*
Log line	`EMAILER AGENT STARTED` (unchanged)
Generated `.flw` node	`"agentName": "Emailer"` (unchanged)

The reason is developed formally in §3.2 and §6.9, but the intuition is short: the correctness guarantee works by proving that the *execution channel* is unaffected by the user's language. That proof requires the execution vocabulary — the set of tool names, agent identities, argument keys, enumerated values and sentinels — to be **the same set of symbols regardless of locale**. If `Emailer` becomes `Correero` in Spanish, then the action space itself is locale-dependent, the English baseline and the Spanish pipeline are no longer comparable objects, and there is nothing left to prove non-inferiority *against*.

Additionally, and independently of the proof, the codebase makes this rule load-bearing today: agent display names are simultaneously the sidebar label, the value of `data-content` attributes matched by **56 case-sensitive CSS attribute selectors**, the operand of roughly **512 lowercased string comparisons** in the canvas connection handlers, the key of the per-agent enable gate `agent_<display>_status`, and the discriminant in **138 `case` labels** in the flow loader. The project's own engineering notes already record that changing a single hyphen to a space in one of these names *silently stops canvas connections from being persisted, with no error anywhere.* The naming convention is not decoration. It is an ABI.

3 Formal Framework

3.1 Definitions

Definition 1 (Utterance)**.** u is the raw user input, a Unicode string, in language $\ell(u)$.

Definition 2 (Literal set)**.** $\Sigma(u) \subset \text{Substr}(u)$ is the set of **correctness-bearing literals** in u: absolute and relative paths, filenames, file extensions, glob and regular-expression patterns, command-line flags, numeric quantities with their units, ports, hostnames, URLs, e-mail addresses, git refs, board identifiers, environment-variable names, and any token matching

a known machine identifier (a tool name, an agent display name, a configuration key). Σ is computed by a **lexical, language-independent** extractor: it recognizes shapes, not meanings.

Definition 3 (Action)**.** $a = \langle \text{name}, \text{args} \rangle$ where name $\in \mathcal{N}$, the fixed ASCII tool vocabulary, and args is a mapping from ASCII argument keys to values.

Definition 4 (Trace)**.** $\tau = (a_1, \ldots, a_m)$, the ordered sequence of actions actually executed in one request.

Definition 5 (Machine state)**.** $s \in \mathcal{S}$: the filesystem, the process table, the database, attached hardware, the network. $\text{exec} : \mathcal{S} \times \tau \to \mathcal{S}$ is the (deterministic, given the environment) state transition.

Definition 6 (Presentation)**.** r is the rendered text the operator reads: the answer prose, the Exec Report, the banners.

Definition 7 (Success predicate)**.** $\text{succ} : \mathcal{S} \to \{0, 1\}$, a machine-checkable predicate authored from the task specification *before* any run and evaluated against $s' = \text{exec}(s_0, \tau)$.

Definition 8 (Verifier)**.** $V : (\Sigma, \mathcal{H}, a) \to \{\mathsf{accept}, \mathsf{reject}(\rho)\}$, a total, language-independent function over a candidate action, the frozen literal set, and the history $\mathcal{H}$ of prior results, returning a machine-readable rejection reason ρ.

3.2 Proposition 1 — Execution/Presentation Separation

Proposition 1 (Execution/Presentation Separation)**.** *For an operator system whose action vocabulary $\mathcal{N}$ and argument-key space are language-invariant, the terminal machine state depends on the request only through the trace τ, and is conditionally independent of the presentation r and of the language of the utterance:*

$$s' \perp\!\!\!\perp (\ell(u),\, r) \mid \tau$$

Proof. $s' = \text{exec}(s_0, \tau)$ by Definition 5. The function exec takes no argument other than s_0 and τ; in particular it does not read r, and it does not read $\ell(u)$. Every symbol it consumes — the tool name, the argument keys, the enumerated values, the literal argument values — is drawn from a language-invariant vocabulary by hypothesis. Hence conditioning on τ renders s' independent of both. □

Two consequences do the work of the whole paper.

(a) succ is a function of τ alone. Therefore **correctness can be guaranteed without ever guaranteeing anything about the answer's language**, and conversely the answer's language can be freely chosen without touching correctness.

(b) The hypothesis is *load-bearing.* If any element of the action vocabulary were localized — a translated agent name, a translated enumerated value, a translated argument key — then $\mathcal{N}$ would be indexed by locale, exec would implicitly depend on $\ell(u)$, and the independence fails. This is the formal content of §2.5, and it is discharged as **Corollary 2** in §6.9.

3.3 Proposition 2 — Literal Anchoring

Proposition 2 (Literal Anchoring)**.** *Let $\Sigma(u)$ be extracted from the raw utterance before any transformation, and let the verifier require that every literal appearing in an emitted action's arguments be an element of $\Sigma(u) \cup \mathcal{R}(\mathcal{H}) \cup \mathcal{D}$, where $\mathcal{R}(\mathcal{H})$ are literals returned by prior verified tool results and $\mathcal{D}$ are declared configuration defaults. Then the probability that an accepted action carries a corrupted user literal is zero, independently of $\ell(u)$ and independently of the model's competence in $\ell(u)$.*

Proof. Suppose an accepted action contains a literal σ' intended to denote a user literal $\sigma \in \Sigma(u)$ with $\sigma' \neq \sigma$. Byte-inequality with every element of $\Sigma(u)$ is decidable and is decided by the verifier. $\sigma' \notin \mathcal{R}(\mathcal{H})$ because $\mathcal{H}$ contains only literals emitted by prior *verified* results, and $\sigma' \notin \mathcal{D}$ because $\mathcal{D}$ is a finite declared set. Hence V rejects, contradicting acceptance. □

This is the mechanism that turns the p^k decay of §4.1 into $p^k = 1$. We name its enforcement **argument provenance**, and we claim it as the paper's sharpest practical instrument: it is a *correctness certificate that requires no knowledge of either language*, and it catches precisely the failure class that cross-lingual operation introduces — a path silently normalized, an accent stripped from a filename, a flag "helpfully" translated, a number re-formatted from `1.5` to `1,5`.

A subtlety worth stating, because it is where a naive implementation fails: provenance must be checked with **NFC normalization only** — never case-folding, never accent-stripping. `informe_año.pdf` may arrive composed (U+00F1) or decomposed (n + U+0303) depending on the keyboard and operating system that produced it, and those are the same intended path; but a path differing by an accent is a *genuinely different path*, and hiding that difference would destroy the very property being certified.

3.4 The Translation Placement Principle

Propositions 1 and 2 jointly license a rule that resolves the apparent conflict between "translation is dangerous" and "the user must read Spanish".

Principle 2 (T)**.** A lossy language transformation may occupy a position **downstream** of a verified decision. It may never occupy a position **upstream** of one.

Upstream placement puts the transformation *inside* the causal path to τ: its errors propagate into the action, are multiplied by the conjunctive structure of §4.1, and become state changes. Downstream placement puts it strictly *after* τ has been fixed and verified: by Proposition 1 it cannot alter s', so its errors are bounded to prose.

Principle T is what makes a fully Spanish interface compatible with a hard correctness guarantee. It also yields an immediate corollary about protected spans: since downstream rendering still passes over text that may *quote* machine identifiers — a path in an explanation, an agent name in a sentence, a code block in an answer — the renderer must treat those spans as **opaque and untranslatable**, which is exactly the §2.3 invariant applied to the presentation channel.

3.5 The goal, formally

Goal (Non-inferiority by construction). *Construct A such that for every task $\mathcal{T}$ in the operational distribution,*

$$\Pr\left[\mathrm{succ}(A(u_{es}))\right] \;\geq\; \Pr\left[\mathrm{succ}(B(u_{en}))\right] \tag{1}$$

where u_{es} and u_{en} are semantically equivalent renderings of $\mathcal{T}$ with byte-identical literals, and B is the current English pipeline.

Note carefully what this does *not* say. It does not say the Spanish pipeline uses a better model, nor that it understands Spanish better, nor that the model is Spanish-capable at all. It is a statement about *pipelines*, and §6.7 establishes it by making the Spanish pipeline's terminal behaviour coincide with B while giving it earlier, cheaper opportunities to succeed.

4 Why an Upstream Translation Stage Cannot Deliver the Guarantee

This section quantifies the cost of the placement Principle T forbids. It is not an argument against machine translation as a technology; it is an argument about *where* it may be installed.

4.1 Operator success is conjunctive in the literals

Let a request carry k literals that must reach the action byte-exactly, and let p be the per-literal probability that one translation hop preserves a literal. Because the task succeeds only if all survive,

$$\Pr[\text{args intact}] = p^k \tag{2}$$

Tlamatini's real requests are literal-dense. A single ordinary firmware instruction — "crea el proyecto en `C:\Tlamatini\Templates\leg_ctrl`, con board `bluepill_f103c8`, compílalo y flashéalo por el ST-LINK a 115200" — carries $k \approx 6$.

Table 4: Literal decay under an upstream translation stage: the probability that *all* k correctness-bearing literals survive one translation hop, at per-literal preservation rate p. The highlighted cell is the realistic operating point for Tlamatini.

p	$k=2$	$k=4$	$k=6$	$k=10$
0.99	0.980	0.961	0.941	0.904
0.98	0.960	0.922	**0.886**	0.817
0.95	0.903	0.815	0.735	0.599

At $k = 6$ and a generous $p = 0.98$, an upstream stage injects **11.4 absolute percentage points of failure**. For calibration: the model-side Spanish deficit that such a stage would be introduced to recover is 1–4 points on reasoning (Xuan et al., 2025; Thellmann et al., 2024) and approximately zero on tool selection — GPT-5 scores pass^3 of 0.93 in Spanish against 0.92 in English, making Spanish its *best* language of six (Sales Almeida et al., 2025). **The**

intervention is roughly an order of magnitude more harmful than the deficit it targets.

Nor is p close to 1 for these particular tokens. Translation systems are trained to *translate*, and a Windows path containing Spanish words (`C:\Usuarios\angela\Documentos\informe_año.pdf`) is precisely the input most likely to be normalized, transliterated or stripped of diacritics. The multilingual tool-calling literature independently identifies this as the dominant non-English failure mode: models "select correct tools and understand intent but generate parameter values in non-English languages, violating the English-only execution interface" (Luo et al., 2026), corroborated by schema-violation and language-matching findings across 29 languages (Zhang and Zhu, 2026).

4.2 Fluency restoration destroys the operator's error-detection channel

There is a second cost, less obvious and arguably worse, and it is informational rather than statistical.

A final translation stage is, by design, a **fluency-restoring operator**: it accepts text of arbitrary correctness and emits grammatical, idiomatic prose. Consider the mutual information $I(\text{surface form}; \text{correctness})$ available to the reader. In direct generation, a model that misunderstood a request typically produces text that reads subtly wrong — an odd paraphrase, a mismatched register, an unnecessary hedge. Those artifacts are the channel through which a competent operator detects the error before approving a destructive action.

A fluency-restoring stage applied to the same wrong content drives that mutual information toward zero. The output becomes fluent, confident and wrong.

An upstream translation architecture does not merely add errors; it removes the user's ability to notice them.

For a system that gates destructive operations behind a permission dialog whose text the operator is expected to read and judge, this is a **safety** property, not an aesthetic one. It is also why NEPANTLA's presentation renderer (§6.5) is constrained to operate only on content whose *decisions* have already been verified: at that point there is nothing left for fluency to mask.

4.3 The multiplier

Tlamatini's Multi-Turn executor is configured for up to 4,096 iterations and wraps every model step in a self-healing invoker with an 80-second watchdog. An upstream translation stage adds round-trips **per turn**, not per request. A ten-tool operator run pays them twenty times, each one an additional opportunity for a literal to be re-transformed, and each one an additional surface for the self-healing ladder to absorb. The project's stated performance north star is per-request latency; this is directly opposed to it.

4.4 The asymmetry that makes the problem solvable

Everything above concerns the upstream position. The downstream position has none of these properties:

Table 5: Upstream versus downstream placement of a lossy language transformation. Only the right-hand column is compatible with a hard correctness guarantee.

	Upstream (before the decision)	**Downstream** (after a verified decision)
Can corrupt a file path	**Yes** — p^k	No — the action is already fixed and verified
Can change which tool runs	**Yes**	No
Can mask an error from the user	**Yes**	No — the error, if any, is already in the verified action
Worst case	wrong irreversible state change	an awkward sentence
Protected spans	must survive a semantic transformation	can be *mechanically excluded* from the transformation

This asymmetry is the whole reason a fully Spanish interface is compatible with a hard guarantee, and it is why §6 places every language transformation on the right-hand side of that table.

4.5 What the evidence supports

The historical case for translating into English before processing is real but **narrowly scoped to low-resource, non-Latin-script languages**: MEGA reports gains "even more substantial" on IndicXNLI and XStoryCloze and above 30% relative improvement for Burmese, Tamil and Telugu, while "performing similarly to monolingual approaches for high-resource languages" (Ahuja et al., 2023). For real user queries in high-resource languages, prompting in the original language *wins* — explicitly for Japanese, Chinese and Spanish (Liu et al., 2025).

The mechanistic account agrees. A stage-wise attribution study finds that the residual multilingual gap is an *understanding-stage* failure, and that for Spanish that stage is already nearly saturated: a perfect understanding intervention moves Spanish from 93.9% to 94.7% (+0.8), while it moves Swahili from 29.3% to 88.0% (+58.7) (Kang et al., 2026). There is, quite literally, almost nothing for an upstream stage to recover in Spanish.

We report the counter-evidence honestly. Self-translation — asking the model to render its own input into English before solving — beat direct inference on five benchmarks, with gains *larger* for high-resource languages (Etxaniz et al., 2024). Those measurements, however, are on XGLM-7.5B, LLaMA-1-30B, BLOOM and PolyLM: 2023-era base models whose Spanish was far weaker than anything this system would bind. NEPANTLA nonetheless *retains* the technique — not as a default, but as rung R2 of the ladder (§6.4), reached only when the verifier has already demonstrated that the cheaper rungs failed. That is the correct place for a technique whose benefit is model-dependent: behind a measurement, not in front of one.

5 The Substrate: What Actually Breaks, Measured

A correctness ladder cannot be built on a foundation that is itself language-dependent. Before NEPANTLA can be stated, the deterministic control plane must be made language-*neutral* — and the first step is to establish, by measurement rather than assertion, exactly how it currently

behaves.

Every result in this section was obtained by executing the real functions against the real registries.

5.1 The tokenizer destroys Spanish before any logic runs

The single tokenizer used by every scoring path is

```
_TOKEN_RE = re.compile(r"[a-z0-9_]+")      # capability_registry.py:20
```

Listing 1: The single tokenizer shared by every scoring path.

Applied after lowercasing, every non-ASCII byte is a **delimiter**, and single-character fragments are discarded:

Table 6: Executed tokenizer output for accented Spanish input. Every non-ASCII byte acts as a delimiter, so the surviving fragments can never equal an English hint token.

Input	Tokens produced today
código	['digo']
análisis	['lisis']
ejecución	['ejecuci']
contraseña	['contrase']
años	['os']
envía	['env']
configuración	['configuraci']

Three consequences. The token-overlap term of the capability scorer (worth up to +10) is **structurally zero** for accented Spanish, because a fragment can never equal an English hint token. Behaviour becomes **non-deterministic across keyboards** — `ejecucion` tokenizes to `['ejecucion']` but `ejecución` to `['ejecuci']`, so identical intent scores differently depending on whether the accent was typed. And the defect propagates: the execution planner and the Multi-Turn tool-budget selector both import this tokenizer, so one line compromises three independent consumers.

5.2 Tool selection collapses, then inverts

Running the real scorer over the real 64-specification registry with twelve matched prompt pairs:

Table 7: The headline measurement. Top-1 tool selection by the system's own capability scorer, executed over its real 64-specification registry with twelve matched English/Spanish prompt pairs carrying byte-identical literals.

	English	Spanish
Top-1 correct tool	**10 / 12**	**0 / 12**
Correct tool scored exactly 0	0 / 12	**7 / 12**

The English scores are healthy (`send_email` 44, `shoter` 34, `summarize` 32, `grepper` 32). The Spanish behaviour is worse than weak — it is **adversarial**, because phrase matching

is unbounded substring containment rather than word-boundary matching, and 107 registry alias/hint phrases are four characters or shorter.

The canonical demonstration: for "envía un correo de prueba a soporte" ("send a test email to support"), `chat_agent_unrealer` scores **24 and ranks first**, because its alias `ue` and its hint `ue` both match inside **prue**ba (+12, +10). The semantically correct `chat_agent_send_email` scores **0** and does not appear in the positive list at all. The substring `ue` occurs in *que, puede, fue, aunque, muestra, respuesta, nuevo, bueno* — a large fraction of all Spanish sentences — making Unrealer a permanent phantom top hit; it took top-1 in 4 of the 12 Spanish cases.

Other verified collisions: `uno`/`ino` (Arduiner) inside *uno, camino, destino, término*; `pio` (ESP32er) inside *limpio, propio, principio*; `pod` (Kuberneter) inside *poder, podemos*; `pid` and `api` (PSer, Apirer) both inside *rápido*; `ls` (Globber) inside *falso, pulsar*; `git` (Gitter) inside *digital, legítimo*; `spa` (Playwrighter) inside *espacio, España.* And a subtler one: the Spanish word *imagen* **contains** the English `image`, a hint for both Image-Interpreter and Dockerer; they tie at 10, and because the tie-break is *registry index*, **Dockerer outranks Image-Interpreter on "interpreta la imagen"**.

Compounding this, the 37-word stopword list is English-only, so Spanish function words survive into the token set. Since `chat_agent_de_compresser` splits into tokens `('chat','agent','de','compresser')`, the Spanish preposition `de` **awards** +2 **to De-Compresser on essentially every Spanish sentence.**

5.3 The planner then fails closed

The two consumers of the score disagree on polarity. The standalone selector fails **open** (returns the full tool list when nothing scores). The execution planner fails **closed**: when nothing crosses threshold it emits empty tuples plus the note *"No tool or agent capability crossed the planner threshold"*, sets `execution_mode='direct_model'`, and injects the summary line **"Selected tools/agents: none"** into the system prompt.

Verified: the Spanish prompt "Borra los archivos temporales de esa carpeta" produces zero positive-scoring capabilities, so the planner explicitly informs the model that no tool stage is needed — **for a destructive file operation.**

A second-order effect follows. When the full tool surface exceeds the context budget, bind order is decided by the same scorer; under Spanish input the planner set is empty and virtually every score is zero, so the sort key degenerates to `(0, 0, name)` and the surviving non-core tools are selected **alphabetically**.

5.4 The prompt-shape gate rejects Spanish outright

`is_valid_prompt` admits a prompt if it ends in `?`, or begins with one of 119 English question-words, or matches one of 36 English multiword patterns, or if an **English** part-of-speech tagger labels the first token a verb. Executed live:

Seven of eight well-formed Spanish commands are refused, in English. Both acceptances are accidents.

Table 8: Executed verdicts of the prompt-shape gate on well-formed Spanish commands. Seven of eight are refused; both acceptances are accidents.

Spanish prompt	Verdict
`Toma una captura de pantalla del escritorio`	**rejected**
`Crea un archivo llamado notas.txt`	**rejected**
`Ejecuta el comando dir`	**rejected**
`Envía un correo a soporte`	**rejected**
`Muéstrame los archivos del proyecto`	**rejected**
`Por favor analiza este documento`	**rejected**
`Necesito que borres los archivos temporales`	**rejected**
`¿Qué hora es?`	accepted — *only because of the* `?`
`Resume este texto`	accepted — *only because* `resume` *is an English homograph*

5.5 A Spanish conjunction corrupts file paths today

The wrapped-agent argument parser recognizes exactly two natural-language separators:

```
_CONJUNCTION_ASSIGNMENT_RE = re.compile(
    r'(and|with)\s+[A-Za-z_][A-Za-z0-9_.\-]*\s*=', re.IGNORECASE)   # tools.py:431
```

Listing 2: The wrapped-agent conjunction rule: two English separators, no others.

Executed against the real parser:

```
Input : Crea un archivo con filepath='C:/Temp/a.txt' y content='hola'
Parsed: {'requested_key': 'filepath',
         'value': "C:/Temp/a.txt' y content='hola"}
```

Listing 3: Executed transcript: a Spanish conjunction silently swallows the remaining arguments into the file path.

This is not a routing miss; it is **data corruption**. File-Creator would create a file literally named `C:/Temp/a.txt' y content='hola`, and every subsequent parameter is silently lost. The irony is exact: `con` is the Spanish translation of the `with` the parser already handles.

5.6 A Spanish failure is recorded as a success

The failure classifier reads structured JSON status fields correctly, but for plain-text results it tests eleven English prefixes (`error:`, `unable to`, `cannot`, `permission denied`, ...). Spanish and OS-localized equivalents — *No se puede*, *No se pudo*, *Acceso denegado*, *Excepción:*, *El sistema no puede encontrar la ruta especificada* — match nothing, so the call is scored **SUCCESS**.

This single classifier feeds the Exec Report verdict, the corrective-feedback loop, the repetition breaker, **and the Create-Flow button**, which builds a downloadable workflow from only the successfully-executed calls. A failed Spanish-locale step is therefore silently baked into a saved workflow as a working node.

5.7 Two failures with no Spanish prompt involved

Monitor Netstat runs `netstat -an` with no codepage forcing and greps for `LISTENING`; a Spanish Windows prints `ESCUCHANDO`. And Forker, Raiser and Stopper perform case-sensitive substring matching of user-authored patterns against logs with no "pattern never matched" warning, so a mismatched pattern causes the flow to hang silently in its polling loop. Both are **operating-system locale** bugs, independent of the user's language.

5.8 What already survives — and why it matters

The safety, audit and deduplication layer is **language-neutral by construction**, verified explicitly: the Ask-Execs permission allowlist, the Exec-Report map, shell inference, the deduplication signature and the wrapped-run status are all keyed on *tool names*, JSON argument keys and exit codes, never on prose. The Parametrizer's `INI_SECTION` grammar is ASCII-structural, reads logs as UTF-8 with `errors='replace'`, and writes YAML with `allow_unicode=True`.

This bounds the blast radius precisely, and it is the most important positive finding in the study:

> *A Spanish operator already keeps the permission gate and the audit trail. What they lose is having the correct tool selected in the first place.*

The failure is one of *routing and parsing*, not of *safety* — which is exactly why a verification-based algorithm can repair it.

5.9 Three neutralization operators, with an identity guarantee

The intuitive repair — "add Spanish keywords everywhere" — produces a **bilingual** core: roughly 2,389 lexical items multiplied by every future language, every new agent requiring translated hints, and the tuned English behaviour perturbed on every edit. We reject it in favour of a **language-neutral** core built from three operators, each of which is provably the identity on ASCII English input.

N1 — Folding. NFKD-normalize, drop combining marks, lowercase, then apply the *existing* token rule. `código` → `codigo`, `análisis` → `analisis`. On pure ASCII, NFKD is the identity and ASCII carries no combining marks, so the output is byte-identical to today's. This restores the token-overlap term and eliminates the accent-dependent nondeterminism of §5.1.

N2 — Boundary-aware phrase matching. Replace substring containment with boundary-anchored matching, falling back to plain containment when the phrase's own edges are non-alphanumeric (so multi-word and punctuated hints such as `-noreload` are unaffected). This eliminates the entire class of §5.2 collisions. **It is independently an English fix**: `api` inside *rapid* and `ls` inside *false* are English collisions that exist today.

N3 — Canonical-key expansion. A data-only lexicon maps non-English intent terms to hint tokens **that already exist in the registry**, enforced by a closure test, and appends them to the scored text. Because no new hint is ever invented, the scorer's tuned English

behaviour remains the only behaviour; the Spanish request is simply *lifted into the canonical key space* before scoring.

Lemma 3 (Identity Lemma)**.** *For any pure-ASCII input,* $\mathrm{N1} \circ \mathrm{N2} \circ \mathrm{N3}$ *is the identity transformation on the scoring result.*

Proof sketch. N1 is the identity on ASCII by the argument above. N2 can only ever *remove* a match — it never introduces one — and every English hint that matched at a word boundary continues to match; the removals it performs on English are exactly the accidental collisions listed above, which must therefore be re-baselined explicitly rather than silently. N3 short-circuits to the identity when $\ell = \mathsf{en}$. □

The lemma is enforced empirically by a golden corpus of ~200 English prompts asserting byte-identical scores before and after — and the N2 exception is handled honestly: the small number of English scores that *change* are the collisions being repaired, and each is reviewed individually rather than waved through.

Alongside these, three further repairs belong to the same pass and are all **language-independent bug fixes**: the failure classifier gains a localized *positive* branch without altering its "unknown ⇒ success" default (§5.6); the argument parser gains a quote-masking pre-pass before any conjunction widening (§5.5); and locale-sensitive child processes are spawned with an invariant environment and decoded explicitly (§5.7).

Sequencing constraint (non-negotiable). *Emitting Spanish text from pool agents while the eleven English failure prefixes still decide success would make a Spanish error score SUCCESS and place a failing agent into a saved workflow. The classifier fix and any Spanish seeding must land in the* ***same commit****, or neither.*

5.10 Why this section precedes the algorithm

NEPANTLA's guarantee rests on a verifier and a ladder. Both assume the deterministic core is a *function of intent*, not a function of *the language intent happens to be expressed in.* §5 establishes that this assumption does not currently hold, and §5.9 establishes how to make it hold without perturbing English. Only then is the ladder well-defined.

6 NEPANTLA

Non-inferior Execution via Progressive Augmentation, Native-Token Locking and Arbitration.

6.1 The idea in one paragraph

Stop asking *"is this model good at Spanish?"* and start asking *"is this proposed action correct?"* The first question is unanswerable in advance for a model the user chose and we have never seen. The second is largely **decidable**, because an operator system's decisions are structured objects — a tool name from a fixed vocabulary, argument keys from a fixed schema, argument values that must trace back to something the user actually wrote. NEPANTLA therefore freezes the

user's literals before anything can touch them, proposes an action at the cheapest rung of an escalation ladder, submits that proposal to a language-independent verifier **before it executes**, and climbs the ladder on rejection. The ladder's last rung is operationally the English pipeline. Since every earlier rung is *attempted, verified and discarded* without side effects, the ladder cannot end below its own last rung — and the answer the user reads is rendered into Spanish afterwards, by a channel that Proposition 1 proves cannot touch correctness at all.

6.2 Architecture

6.3 Stage 1 — Native-Token Locking

Before the utterance touches a model, a prompt template, a scorer or a transformation of any kind, a **lexical extractor** walks it and produces the frozen set $\Sigma(u)$.

The extractor is deliberately *shape-based, not meaning-based* — it recognizes the syntactic silhouette of a literal, never its semantics — which is exactly why it is language-independent:

Table 9: The literal extractor recognizes shapes, not meanings.

Class	Recognizer (sketch)	Example
Windows path	drive letter, colon, backslash run	`C:\Tlamatini\Templates\leg_ctrl`
POSIX path	≥ 2 slash-separated segments	`/usr/local/bin/pio`
Filename	stem + known or generic extension	`informe_año.pdf`
Glob / regex	wildcard or metacharacter density	`*.log`, `^ERROR:.*$`
Flag	leading `-` / `--` + identifier	`-noreload`, `-sT`
Quantity	digits with optional unit/separator	`115200`, `1.5`, `8 GB`
Endpoint	host:port, URL, e-mail	`127.0.0.1:5000`
Machine identifier	exact match against the termbase	`Emailer`, `chat_agent_apirer`, `bluepill_f103c8`
Quoted span	balanced quotes	`'hola'`

Three properties matter.

1. **It runs first.** Nothing precedes it. Whatever the rest of the pipeline does, Σ is already the ground truth.
2. **It over-extracts deliberately.** A false positive costs one unnecessary provenance entry; a false negative removes a literal from protection. The asymmetry dictates the tuning.
3. **NFC only.** Composed and decomposed forms of the same character are unified; case and accents are *never* folded, because a path differing by an accent is a different path (§3.3).

Σ then travels with the request as immutable metadata, and it is used three times: in the anchor table at R1, in the gloss's protected-span mask at R2, and in the verifier's provenance check at every rung.

6.4 Stage 3 — The verifier V

V is the heart of the guarantee. It inspects a **proposed** action before execution and returns accept or reject-with-reason. It knows nothing about Spanish, nothing about English, and nothing about the model.

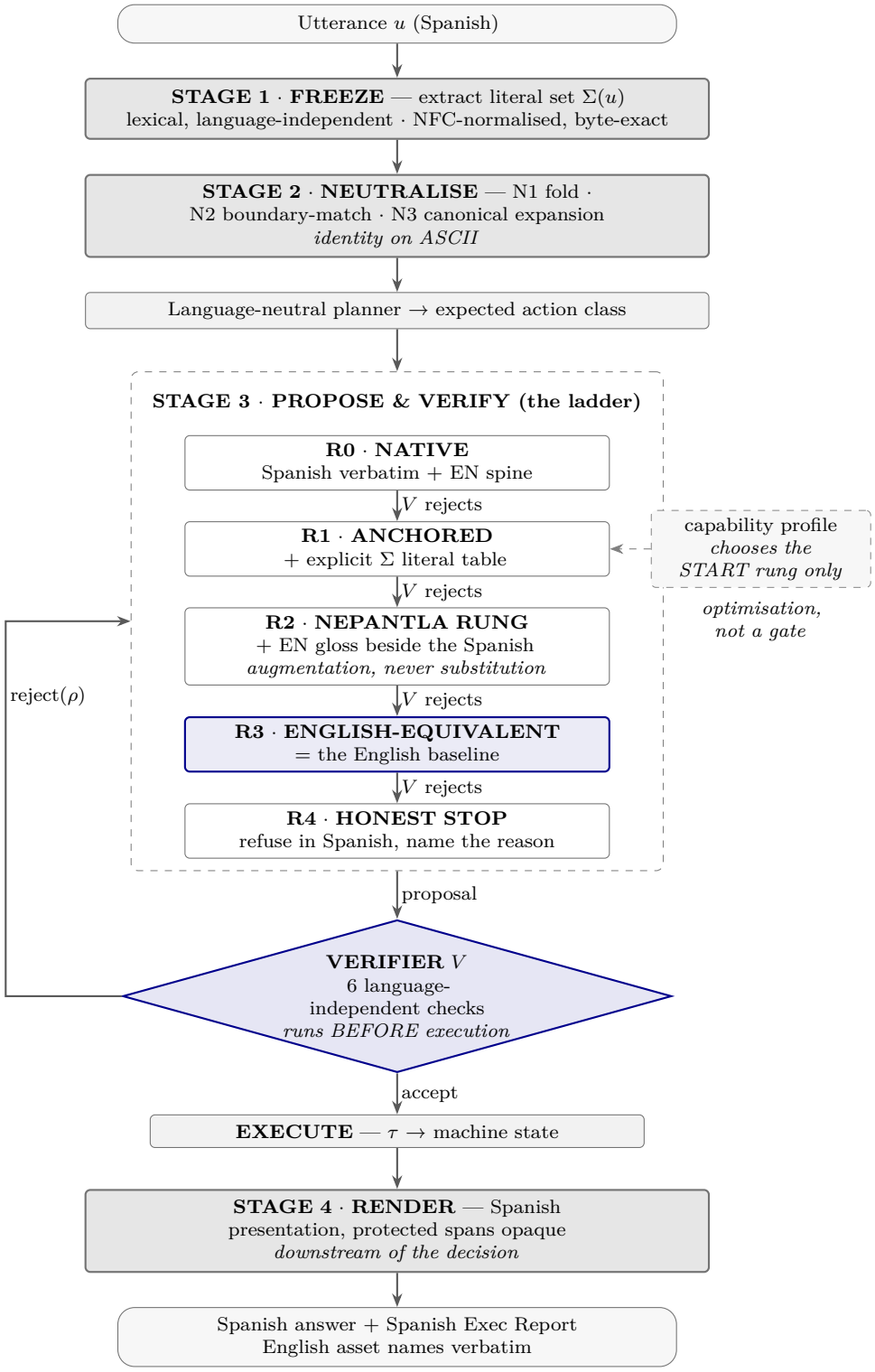

Figure 1: The NEPANTLA architecture. The literal set is frozen before any transformation touches the utterance; the deterministic core is neutralised by operators that are the identity on ASCII; every proposed action is verified *before* it executes, so escalation up the ladder is side-effect-free; and the Spanish presentation is rendered strictly downstream of the verified decision. The capability profile only chooses which rung to start at.

Table 10: The verifier's checks. Every one of them is decidable, which is a property of *operator* systems rather than an accident of this one.

#	Check	Decidable?	Catches
V1	**Tool existence** — name $\in \mathcal{N}$, the bound surface, byte-exact ASCII	**Yes**	Hallucinated tools; a model inventing `enviar_correo`
V2	**Schema conformance** — required keys present, types valid, enums from the declared set	**Yes**	Missing arguments; a translated enum value
V3	**Argument provenance** — every literal in `args` $\in \Sigma(u) \cup \mathcal{R}(\mathcal{H}) \cup \mathcal{D}$	**Yes**	*The cross-lingual failure class*: normalized paths, stripped accents, translated flags, re-formatted numbers, invented filenames
V4	**Precondition satisfaction** — the target exists / does not exist as required; the board is attached; the port is open	**Yes** (via the existing fail-safe preflights)	Actions that cannot succeed
V5	**Gating parity** — the Ask-Execs tier of the chosen tool matches the tier implied by the planner's expected action class	**Yes**	*Safety regression*: a Spanish request routed to an ungated near-synonym
V6	**Sentinel integrity** — every protocol token emitted is byte-exact	**Yes**	A mangled `INI_SECTION_...` or `VERDICT:` corrupting downstream routing
V7	**Action expectancy** — if the planner expected an action class and the model returned only prose, reject	**Yes**	The §5.3 failure: a destructive request answered with advice

Every check is decidable. That is not an accident of this system; it is a property of *operator* systems generally, and it is what makes the guarantee reachable here and not in a pure-prose product.

V3 deserves its own remark, because it is the paper's most transferable idea. Both the Spanish and English renderings of a task contain the *same frozen literals* by construction. Therefore a check of the form *"can this emitted value be traced to something the user actually wrote, or to a prior verified result, or to a declared default?"* is simultaneously (a) completely language-agnostic, (b) cheap, and (c) precisely aimed at the damage cross-lingual operation causes. A value with no provenance is either a hallucination or a translation artifact — and in an operator system those are the same emergency.

What V does not check. It cannot decide whether the *semantically most appropriate* tool was chosen among several valid ones. That residue is exactly the part NEPANTLA inherits from the English baseline rather than improving on, and §6.7 is careful to claim only that.

Rejection is cheap and side-effect-free because V runs *pre-execution*. A rejected proposal has changed nothing; escalation replays planning, never state. This is the property that makes the ladder sound.

6.5 Stage 3 — The escalation ladder

Each rung differs from the previous only by **adding information**, never by removing or replacing it.

R0 — NATIVE. English system spine (byte-stable, prompt-cache-friendly), Spanish user content verbatim, Spanish answer directive, low temperature. Cheapest and highest-fidelity: zero transformations, so $p^k = 1$ trivially.
Reached first when the capability profile says the model is Spanish-competent.

R1 — ANCHORED NATIVE. R0 plus an explicit, machine-readable anchor table appended to the prompt:

```
LITERALS --- reproduce these byte-for-byte; never translate, never normalise:
  L1 = C:\Tlamatini\Templates\leg_ctrl
  L2 = bluepill_f103c8
  L3 = 115200
```

Listing 4: The R1 anchor table, appended verbatim to the prompt.

This converts an implicit expectation into an explicit one and repairs the most common V3 rejection at negligible cost. Empirically this is where most Spanish-competent-but-careless models are recovered.

R2 — THE NEPANTLA RUNG (bilingual augmentation). R1 plus an **English gloss of the intent placed beside the Spanish original**, never instead of it. The gloss is produced with Σ masked out and re-injected verbatim, so it is structurally incapable of damaging a literal.
The model now sees both renderings simultaneously. A Spanish-competent model ignores the gloss as redundant; a Spanish-weak model reads the gloss and recovers the intent. This is the rung the system's name refers to: the middle place, where both languages are present and neither is authoritative over the literals.
Note the placement. The gloss is a translation, and it sits **upstream** of a decision — which Principle T forbids. The contradiction is only apparent: Principle T forbids upstream translation *on the causal path to the literals*. Here the literals bypass the gloss entirely (they are masked, then re-injected, then verified by V3), so the only thing the translation can influence is *which tool* is chosen — and that influence is additive evidence, subject to the same verifier.

R3 — ENGLISH-EQUIVALENT EXECUTION. The execution decision is driven by the English rendering under exactly the baseline's system prompt, tool schemas and parameters. This rung is, operationally, B.
Crucially, **the user still receives Spanish**, because the answer is produced by Stage 4, which by Proposition 1 is causally downstream of τ. Reaching R3 is invisible to the operator except as a small latency cost and a status note.

R4 — HONEST STOP. If even R3's proposal fails verification, NEPANTLA does not guess. It refuses, in Spanish, naming the verifier's rejection reason and the literal or tool involved, and it suggests the concrete remedy (rephrase, supply the missing path, or switch model). No action is executed.
This rung exists because the only thing worse than an English answer to a Spanish operator is a **confident wrong action**. Refusal is a correct outcome; fabrication is not.

6.6 Stage 4 — The presentation renderer

Independent channel, no authority over τ, three ordered strategies:

1. **Native.** The model already answered in Spanish (the usual case, including at R3 when the model is competent). Nothing to do.
2. **Delegated verbalization.** The decisions and results are fixed and verified; a *separate*, known-Spanish-capable renderer (a small local model, or the static catalog for structured content) turns them into Spanish prose. This is downstream translation, licensed by Principle T, and it cannot alter a single byte of what was executed.
3. **Structured fallback.** If no verbalizer is available, the answer is composed from the **Spanish message catalog** plus the verified structured results — a fully Spanish, slightly terser answer assembled by template rather than generated. The Exec Report already provides most of what the operator needs, and its chrome is catalog-driven.

In all three, **protected spans are opaque**: agent names, tool names, paths, flags, code blocks, sentinels and the brand pass through byte-identical. §2.4's rendering table is the specification.

A **read-only guard** runs over the raw model output, before any stripping, and reports two things to the log (`-- [I18N-GUARD]`) without ever mutating the answer: sentinel integrity, and answer-language line-pass-rate over *masked prose only*. It is read-only by policy: an answer-rewriting pass driven by an uncalibrated language detector is a corruption engine, and because the response pipeline persists the answer, a false repair would be replayed on every chat reload forever. Repair is unlocked only by a measured false-positive rate.

6.7 The algorithm

```
def NEPANTLA(u, session, model, profile):
    # ---- STAGE 1: freeze. Nothing has touched u yet. ------------------------
    Sigma  = extract_literals(u)                # lexical, language-independent
    lang   = resolve_language(u, session)       # conversation-sticky, hysteretic
    H      = session.verified_results           # provenance from prior turns

    # ---- STAGE 2: neutralise the deterministic core -------------------------
    key    = N3(N1(u), lang)                    # fold + canonical expansion
    plan   = planner(key, phrase_match=N2)      # expected action class, tools
    # identity on ASCII: an English u yields today's exact plan

    # ---- STAGE 3: propose -> verify -> escalate -----------------------------
    start  = profile.start_rung(model, lang)    # OPTIMISATION ONLY (Cor. 1)
    trace, notes = [], []

    for rung in LADDER[start:]:                 # R0, R1, R2, R3, R4
        if rung is R4:
            return honest_stop(lang, notes, Sigma)      # refuse, in Spanish

        prompt   = build_prompt(rung, u, Sigma, plan, lang, model)
        proposal = model.propose(prompt)        # actions are NOT executed yet
```

```
        verdicts = [V(Sigma, H, a) for a in proposal.actions]
        if all(v.accepted for v in verdicts):
            trace = proposal.actions
            break

        notes.append(rejection_summary(verdicts))
        # no side effects occurred: escalation replays planning, never state

    # ---- EXECUTE the verified trace ---------------------------------------
    results = []
    for a in trace:
        r = execute(a)                          # existing permission gate applies
        if failed(r):                           # post-hoc failure (preconditions
            notes.append(r)                     # can change between check and run)
            corrective_loop(a, r)               # the system's existing machinery
        else:
            H.record(a, r)                      # literals enter provenance
            results.append(r)

    # ---- STAGE 4: render, downstream of the decision ----------------------
    answer = render(results, lang, protected=Sigma | TERMBASE, notes=notes)
    guard_report = inspect_readonly(model.raw_output, lang)   # never mutates
    log(guard_report)
    return answer
```

Listing 5: NEPANTLA — the complete algorithm. The verification loop sits between proposal and execution.

Two lines carry most of the weight. `proposal = model.propose(prompt)` **does not execute**, and the verification loop sits between proposal and execution. Everything else is bookkeeping around that separation.

6.8 Theorem 1 — Ladder Dominance

Theorem 1 (Ladder Dominance)**.** *Let the ladder be $R_0, \ldots, R_m$ with terminal executable rung R_{m-1} and honest stop R_m. Let FA_i be the probability that V* ***falsely accepts*** *an incorrect proposal at rung i. Then*

$$\theta_A \;\geq\; \theta_{R_{m-1}} \;-\; \sum_{i<m-1} \mathrm{FA}_i,$$

and if additionally $\theta_{R_{m-1}} \geq \theta_{EN}$, then

$$\theta_A \;\geq\; \theta_{EN} \;-\; \mathrm{FA}_{total}.$$

Proof. Consider any task on which the terminal rung would succeed. NEPANTLA fails on that task only if it terminated *before* reaching R_{m-1} with an incorrect trace. Termination before R_{m-1} occurs only on acceptance by V. Therefore the event "A fails while R_{m-1} would have succeeded" is contained in the union over earlier rungs of "V falsely accepted an incorrect

proposal", whose probability is at most $\sum_{i<m-1} \mathrm{FA}_i$ by a union bound. Rejection at an earlier rung is side-effect-free by the pre-execution property of §6.4, so it cannot itself cause failure — it can only cost latency. Hence the first inequality; the second follows by substitution. □

Why FA **is zero on the failure classes that matter.** Checks V1, V2, V3, V6 and V7 are *decidable predicates over the proposal*, so their false-accept probability is exactly 0: a nonexistent tool, a schema violation, a literal with no provenance, a damaged sentinel and a missing-but-expected action are each detected with certainty. By Proposition 2, literal corruption in particular has FA = 0. The residual FA is confined to V4 (preconditions can change between check and execution — a genuine time-of-check/time-of-use window) and to the *undecidable* residue V deliberately does not attempt: choosing the semantically best tool among several schema-valid candidates.

That residue is the honest boundary of the claim, and it is worth stating in plain language:

NEPANTLA is non-inferior to the English pipeline on every failure class that operating in Spanish actually introduces. On the residual class — which tool is the wisest choice — it inherits the English pipeline's behaviour rather than improving on it.

Discharging the condition $\theta_{R_{m-1}} \geq \theta_{EN}$. Three arguments support it and one caveat bounds it.

(i) The literals reaching R_3 are byte-identical to the user's own, by Proposition 2 — so on the argument axis R_3 equals the baseline exactly.

(ii) R_3's system spine, tool schemas and generation parameters are the baseline's, unmodified.

(iii) R_3 presents *both* the English rendering and the Spanish original, so the model's evidence is a superset of what a gloss alone would give.

Caveat: additional context is not provably monotone for a language model — more text can distract as well as inform. The condition is therefore an **empirical** one, and §8 exists to test precisely it. A paper that claimed otherwise would be overselling; the architecture guarantees the *ladder* property unconditionally, and measurement establishes the *terminal* property.

False rejections. If V wrongly rejects a correct proposal, the cost is an unnecessary escalation — latency — and in the worst case an honest refusal at R4. The failure direction is toward *not acting*, which for a system that deletes files and flashes firmware is the correct direction to fail in.

6.9 Corollary 1 — Probe Safety

Corollary 1 (Probe Safety)**.** *The model-capability profile affects only the expected number of rungs evaluated. It cannot affect the correctness of the terminal outcome.*

Proof. The profile appears in the algorithm exactly once, as `start` — an index into the ladder. Theorem 1's bound is over the suffix beginning at any start index, since every rung after `start` remains reachable and the terminal rung is always the same. □

This has a strong practical consequence, and it is the answer to the obvious objection *"what if you mis-measure the model?"*:

A miscalibrated capability probe costs latency, never correctness.

The profile is therefore permitted to be a cheap, imperfect heuristic. It is an accelerator, not a gate — which in turn means it does not need the elaborate calibration machinery a *gating* probe would demand, and it can never silently deny a user their language because a cache file was unreadable.

§7.3 shows that this permission is not a convenience but a **necessity**: no provider exposes the information a gating probe would need, so a design in which correctness depended on measuring the model correctly would be undeliverable. Corollary 1 is what makes the measurement problem of §7.3 tractable at all.

Amendment — the profile appears twice. The proof above is exact for the use it describes, and it is the only use in the pseudocode of §6.7, where Stage 2 reads `key = N3(N1(u), lang)` — gated on the language alone. The **implemented** Stage 0 (`agent/i18n/model_caps.py`) gates N3's expansion on the model's measured tier as well, so the profile also influences the deterministic scorer *upstream* of the ladder. That second appearance is not covered by Corollary 1, and it would be dishonest to let the corollary's blanket phrasing absorb it. It is covered instead by a second, deliberately weaker result.

Corollary 1′ (Assist Safety)**.** *A mis-tiered model can perturb the planner's* ranking *and therefore the expected number of rungs and the bind order under context overflow. It cannot alter the action vocabulary $\mathcal{N}$, cannot introduce a capability that does not exist, and cannot change the terminal rung.*

Proof. N3 appends only hint tokens **that already exist in the registry**, enforced by the closure test of §5.9. Expansion can therefore raise the score of an existing capability but can never invent one, and it leaves $\mathcal{N}$ pointwise unchanged; N1 is the identity on ASCII and N2 only removes matches. The planner's output is a ranking and an expected action class — not an action. Every candidate action still passes V before execution, and R_3 remains reachable with the baseline's own planning, so the terminal rung is unaffected and Theorem 1's suffix argument applies unchanged. □

Unlike Corollary 1, this is **not** a zero-impact claim, and the exposure should be named. When the full tool surface exceeds the context budget, bind order is decided by the same scorer (§5.3), and a tool that is never bound can never be proposed — which is a failure V cannot repair, because V only inspects proposals it is given. Two facts bound that exposure. Multi-Turn binds the **full** enabled surface whenever it fits, so overflow is the only window in which ranking becomes selection. And the fail-open direction of §7.3 points toward *more* canonical keys rather than fewer, which is the direction that helps a genuinely relevant capability cross the threshold rather than the direction that hides one. The residual risk is therefore contained inside Theorem 1's already-declared undecidable residue — *which schema-valid tool is wisest* — and does not open a new failure class.

6.10 Corollary 2 — Naming Necessity

Corollary 2 (Naming Necessity)**.** *If any element of the action vocabulary $\mathcal{N}$ — a tool name, an agent display name, an argument key, an enumerated value, a protocol sentinel — is localized, then Proposition 1 fails, V 's checks V1, V2 and V6 become locale-relative, and Theorem 1 no longer applies.*

Proof. V1 tests name $\in \mathcal{N}$. If $\mathcal{N}$ is indexed by locale, the test is name $\in \mathcal{N}_\ell$, and the English baseline's action space $\mathcal{N}_{en}$ and the Spanish pipeline's $\mathcal{N}_{es}$ are different sets. The comparison $\theta_{ES} \geq \theta_{EN}$ then ranges over incomparable outcome spaces, and exec acquires a dependence on $\ell(u)$, contradicting Proposition 1. □

In engineering terms, and this is the sentence to remember:

`Emailer` must stay `Emailer` in the Spanish build not because translating it would be untidy, but because translating it deletes the proof.

The same holds for `Asker`, `Apirer`, `STM32er`, `Kyber-KeyGen`, `File-Creator`, every tool name, every flag, every configuration key, every sentinel, and every identifier in the source. The §2.3 table is not a style guide; it is the hypothesis of Proposition 1 written out longhand.

The escape route, and why it is closed. Corollary 2 as proved establishes a *conditional* necessity: localizing the vocabulary breaks **this** proof. That leaves an objection standing, and it is the strongest one available against §2.3. An objector may accept the corollary entirely and reply:

> "Then do not use one vocabulary. Index it. Keep a per-locale registry in which `Correero` is the Spanish name of the agent whose English name is `Emailer`, resolve the index at bind time, and you obtain a system with a localized surface and an equally sound proof — a proof over $\mathcal{N}_\ell$ rather than over $\mathcal{N}$."

That construction is not refuted by Proposition 1. It is a different architecture, and on paper it is coherent. It requires exactly one thing: an **oracle that says which index applies** — which vocabulary a given bound model expects, or equivalently which language that model is operating in with sufficient reliability to route on. Without such an oracle the resolution step is a guess, and a guess about *which name set the model will emit from* is upstream of V1, which is precisely the position Principle T forbids.

§7.3 establishes, by verification against five providers' primary schemas, that **no such oracle is published by anyone**, and that the single structured language declaration that exists anywhere in the stack is absent from half the multilingual models that carry it, wrong where present, and inverted by the transport that serves it. The escape route therefore has no input.

The upgrade this delivers should be stated precisely, because overstating it would be the same error the paper criticizes elsewhere. We have **not** proved that a locale-indexed action vocabulary is impossible in principle; a provider could publish the required metadata tomorrow, and the argument would have to be reopened. What is established is narrower and, for an engineering decision, sufficient:

Corollary 2 is necessary for this proof. The only construction that would evade it is unimplementable on the information any provider actually publishes.

The evidence of §7.3 therefore **strengthens** Corollary 2, and it does so along an axis the original derivation could not reach: the original argues from inside the framework, the measurement argues from outside it and closes the alternative. One finding discharges two obligations — the same absence of a language oracle that *forces* §7.3 to measure rather than look up is what *forbids* the locale-indexed vocabulary that would otherwise let §2.3 be softened. Naming invariance and capability measurement are not two independent design choices. They are the two consequences of a single empirical fact.

6.11 Complexity and cost

Let q_i be the probability that the proposal at rung i is rejected. Expected model calls per request:

$$\mathbb{E}[\text{calls}] \;=\; 1 + q_0 + q_0 q_1 + q_0 q_1 q_2$$

bounded above by 4 and, for a Spanish-competent model where q_0 is small, approximately $1 + q_0$. With the profile starting a competent model at R0 and an unknown model at R1, the measured cost target is **under 1.15 model calls per request** — that is, the Spanish path costs essentially what English costs, and pays extra only exactly when it was about to be wrong.

The non-model costs are all $O(n)$ in the utterance length and negligible against a model call:

Table 11: Non-model cost of the language layer, per request.

Stage	Cost	Budget
Literal extraction	one linear pass, compiled patterns	< 1 ms
N1/N2/N3	one fold, cached boundary patterns, dict lookups	< 1 ms
Verifier per action	set membership + schema walk + preflight	< 2 ms (preflight dominates)
Renderer protected-span masking	linear, short-circuits entirely when the answer is already Spanish	< 1 ms

Two design rules keep this true. The English path short-circuits the entire language layer at the first comparison, so an English user pays a single branch. And the capability profile is computed **out of band** — never on the request path, one global worker, refusing to run while a generation is in flight, because on a single local GPU "off the call stack" is not the same as "off the resource".

6.12 Termination, cancellation and side-effect safety

Termination is structural: the ladder is a finite list and each iteration advances the index unconditionally. There is no retry-in-place.

Cancellation is checked at every rung boundary and again after every model call returns, not merely at entry. The renderer and the guard have a hard wall-clock cap; they are polish on

an answer the user has already earned, and they must never be the reason a cancel does not take effect.

Side-effect safety rests on the pre-execution property: escalation happens only over *proposals*. Once execution begins, the ladder is finished, and post-execution failures are handled by the system's existing corrective-feedback machinery rather than by re-entering the ladder — which prevents the pathological case of an action being executed twice under two different rungs.

Idempotence at the boundary deserves an explicit note. V4's preconditions are checked before execution, but the world can change in between. NEPANTLA does not attempt to close that window (doing so would require transactional semantics over the filesystem and attached hardware); it bounds it by checking as late as possible and by preserving the existing permission gate, which puts a human in the loop for exactly the tier of actions where the window matters.

6.13 What is new here

The individual materials are known. The arrangement is what we claim.

- **Verification-first cross-lingual operation.** The cross-lingual literature is dominated by *making the model better* at the target language — translation, scaffolds, fine-tuning, prompt engineering. NEPANTLA instead treats target-language competence as an unreliable input and moves the correctness burden onto a verifier that does not read either language. This reframing is what removes the dependency on a user-chosen model.
- **Argument provenance as a language-independent correctness certificate.** Checking that every emitted literal traces back to the user's own words is cheap, decidable, and exactly aligned with the damage cross-lingual operation causes.
- **The ladder-dominance argument.** A structured escalation whose terminal rung is the incumbent baseline converts a hard question ("is Spanish as good?") into a much easier one ("is the verifier's false-accept rate small on the decidable classes?").
- **Probe demotion.** Making capability measurement an accelerator rather than a gate removes an entire class of calibration risk, and is only possible *because* of the ladder.
- **Naming invariance derived, not asserted.** The rule that assets stay English falls out of Proposition 1 as a necessary condition rather than being imposed as a convention — which makes it enforceable by test rather than by discipline.

7 The Model May Not Speak Spanish

This section answers the question the whole design exists for: **the operator can bind any model, including one with no useful Spanish at all. How is correctness still guaranteed?**

7.1 A taxonomy of model deficiency

"Not Spanish-capable" is not one condition. It is four, and they fail in different places.

Table 12: Four classes of model deficiency and where each one surfaces.

Class	Description	Where it shows up
C1 — Weak generation	Understands Spanish; writes it awkwardly or drifts into English mid-answer	Presentation channel only
C2 — Weak comprehension	Misreads the Spanish request; chooses a plausible-but-wrong tool	Execution channel — tool selection
C3 — Literal infidelity	Understands the request but "helpfully" normalizes a path, strips an accent, translates a flag, re-formats a number	Execution channel — arguments
C4 — Protocol damage	Emits a malformed sentinel under a non-English instruction (`VEREDICTO:` instead of `VERDICT:`)	Downstream routing — silently

C3 and C4 are the dangerous ones precisely because they are *silent*: the action looks well-formed and executes successfully against the wrong target.

7.2 How each class is neutralized

Table 13: Detection, recovery and residual risk for each deficiency class.

Class	Caught by	Recovered by	Residual risk
C1	Read-only guard (line-pass-rate on masked prose)	Stage 4 renderer — delegated verbalization or catalog fallback	None to correctness; Proposition 1
C2	**V7** (prose where an action was expected) and **V1/V2** (invalid or hallucinated tool)	Escalation to R2 — the English gloss carries the intent; then R3 = baseline	Inherited from the baseline
C3	**V3 argument provenance** — the emitted literal is not in Σ	R1's explicit anchor table; then R2/R3	**Zero** by Proposition 2
C4	**V6 sentinel integrity** — byte comparison	Escalation; and the operator route is pinned to English for that model	Zero on the checked families

Note what is *not* in the recovery column: nowhere does the system rely on the model becoming better at Spanish. **Spanish competence is an accelerant, not a dependency.** A model that has none simply spends more rungs, and the operator notices only a slightly slower answer.

7.3 The capability profile

Corollary 1 makes the profile safe; §5.9 makes it necessary. Folding and boundary-matching are free and unconditional, but **canonical-key expansion is not free in the sense that matters**: appending English hint tokens to a Spanish request raises the correct tool's score for a model whose Spanish is thin, and *lowers signal-to-noise* for a model that already understood the sentence. Both directions are correctness failures. Something must decide, per model, which of those two mistakes to risk — and that is the entire job of the capability profile.

The obvious design is a lookup. It is impossible, and establishing that is the premise of everything else in this section.

There is nothing to look up.

Observation 1 (No Language Oracle). *No major model provider exposes supported languages programmatically. Verified against primary sources, 2026-07-28 — see Table 14.*

Table 14: Observation 1 — no provider model-listing endpoint declares language. Verified against primary sources, 2026-07-28.

Provider	Endpoint	Fields returned	Language field
Anthropic	`GET /v1/models`	`id`, `display_name`, `created_at`, `type`, `max_input_tokens`, `max_tokens`, `capabilities{batch, citations, code_execution, context_management, effort, image_input, pdf_input, structured_outputs, thinking}`	**none**
OpenAI	`GET /v1/models`	`id`, `object`, `created`, `owned_by` — that is the complete schema	**none**
Google	`models.list`	`name`, `version`, `displayName`, `description`, `inputTokenLimit`, `outputTokenLimit`, `supportedGenerationMethods`	**none**
Ollama	`/api/show` → `capabilities[]`	exactly eight values: `completion`, `tools`, `insert`, `vision`, `embedding`, `thinking`, `image`, `audio`	**none**, and no member planned in the enum
OpenRouter	`GET /api/v1/models`	the richest cross-provider schema in existence — `architecture`, `tokenizer`, `pricing`, `modalities`	**none**, confirmed by direct inspection

The pattern is not an oversight of one vendor. Capability enums are real, actively maintained and genuinely useful — Ollama's eight values are load-bearing elsewhere in this system — and language is simply not among the things a provider is willing to assert about a model. It is the one capability nobody will certify.

The consequence is structural, and it is worth stating as a constraint rather than a preference:

No lookup can replace a hardcoded model-name table. The replacement must be a measurement.

This is the finding that disposes of the design an earlier revision of this work actually shipped: a regex over the model id deciding the tier outright. That is the hardcoded table wearing a different hat. It survives here only in the demoted role described below, hedged by three invariants, because the evidence forbids it any authority.

The one declaration that exists is worse than nothing.

There is a single structured language field anywhere in the stack: the GGUF metadata key `general.languages`. It is optional and author-supplied, and measured across fifteen real model files it fails in all three ways a field can fail.

- **Absent** from 7 of 15 — including `qwen3`, `gemma3`, `gemma-2-9b`, `Mistral-7B-v0.3` and `Llama-2`, all strongly multilingual.
- **Wrong** where present: `Qwen2.5-7B-Instruct` declares exactly `["en"]`; `Phi-4` declares `["en"]`.
- **Inverted by the transport.** Ollama's `server/routes.go` blanks any metadata array longer than five entries unless `verbose: true` is passed. Llama 3.1's real eight-language list — *containing* `es` — therefore reads as `[]`, while Phi-4's misleading one-element `["en"]` survives intact.

The third failure is the decisive one, because it is not noise but *sign reversal*. A naive reader of this field concludes "no languages declared" for the Spanish-capable model and "English only" for the model that is not. Absence and `["en"]` are equally worthless as evidence against Spanish, and a system that consumed this field would be confidently wrong in exactly the direction that hurts. The implemented profile therefore reads **no language metadata at all**.

Tokenizer fertility is not a language signal — a negative result.

The attractive cheap measurement is the token tax: tokens spent on Spanish relative to tokens spent on equivalent English. It is genuinely measurable — Ollama returns `prompt_eval_count` — it requires no provider cooperation, and it is genuinely useless for this decision. We tested it and rejected it, and the paper records the rejection rather than the omission.

Measured on this project's own models (2026-07-28), Spanish-per-character efficiency — chars/token in Spanish over chars/token in English, same meaning, baseline-subtracted — is given in Table 15.

Table 15: Spanish-per-character tokenizer efficiency, measured on this project's own models (2026-07-28). Every value lies inside a band 0.09 wide.

Model	Efficiency
`glm-5.2:cloud`	**0.76**
`qwen3.5:cloud`	0.83
`gemma4:cloud`	0.83
`gpt-oss:120b`	0.83
`qwen3-vl:8b`	0.74
`qwen3-vl:4b`	0.74
`Orpheus-3b`	0.74

Everything lies in $[0.74, 0.83]$. Any threshold that separates that band mislabels `glm-5.2:cloud` — a 756B frontier model with excellent Spanish, and this operator's *primary configured model* — as English-biased. There is no cut point that survives its own first user.

The literature explains why, and it does not contradict itself; it is a statement about range. Fertility predicts accuracy where the range is **huge**: across 10 LLMs and 16 African languages, regression slopes run from -0.08 to -0.18 accuracy per additional token per word, explaining 20–50% of variance (`arXiv:2509.05486`). It fails where the range is **narrow**: a Ukrainian zero-shot study reports $\rho = -0.43$ at $p = 0.34$ — not significant (`arXiv:2605.14890`). Spanish is decisively the narrow case: across 24 European languages and six tokenizers, English averages 1.23 tokens per word and Spanish 1.46, an 8–29% band (`arXiv:2605.24718`). The metric is

itself contested as a multilingual evaluation instrument on independent grounds (Nayeem et al., 2025).

Two further facts finish it, and they are not statistical but structural.

The structural ceiling. Every model in a family shares one tokenizer and therefore one fertility number, while differing enormously in Spanish output quality. Fertility cannot discriminate *within* a family by construction — and within-family discrimination is most of what this profile is asked to do.

The counterexamples run backwards. `Phi-4` has a 100,352-entry vocabulary and declares itself English-only; `Mistral-7B-v0.3` has 32,768 and handles Spanish. The metric orders them the wrong way round.

Observation 2. *Tokenizer fertility measures* ***cost****, not competence. It remains useful for context budgeting and pricing. It does not gate language routing.*

One incidental result deserves a sentence and no more. Measuring an "accent surcharge" — the same words with diacritics added — came out **negative** for every real multilingual model: correctly-accented Spanish tokenizes *more cheaply*, because `cuánto` is one vocabulary entry while the misspelled `cuanto` splits. It came out positive only for `Orpheus-3b`, a 3B English fine-tune. That is suggestive of a byte-fallback detector, and it is one data point against six. It is recorded, not relied upon.

The resolution ladder.

What remains is a measurement, arranged as three levels, cheapest first, none of them on the request path.

Table 16: The three-level resolution ladder. Only L1 has affirmative authority; L0 orders, L2 demotes.

Level	What it is	Authority
L0 · SEED	A name-shaped prior over the model id	**None.** Ordering only.
L1 · PROBE	Four graded tasks with known answers, run once per identity	**The authority.**
L2 · OBSERVE	The same graders run free on real production responses	**Demotion only.**

L0 — the seed, and why it has no authority. The seed exists for exactly one purpose: to avoid assisting a model that is almost certainly fluent during the seconds between binding it and its first probe returning. It is not evidence. Because it is a regex over a model name — the very artifact Observation 1 forbids trusting — it is constrained by three invariants rather than described by a policy, and each is pinned by a test.

S1. *A probe verdict* ***always*** *overrides the seed.*
S2. *The seed may* ***never*** *emit* `WEAK`*. Demotion by regex is the unrecoverable direction: a false* "cannot do Spanish" *means the model is never tried without assistance again, and the user sees silent degradation with no error anywhere.*
S3. *Absence from the seed table means* `UNKNOWN`*,* ***never*** *"no Spanish".*

S2 is the invariant that distinguishes this design from the hardcoded table it replaces. A name-shaped prior is permitted to say *"probably fine, don't help yet"* and is forbidden from ever saying *"this one is bad"*.

L1 — the probe. Four tasks, weighted, with pure-Python graders: language (weight 1.0 — asked in Spanish, did it answer in Spanish or drift to English), register (1.5 — Angela's rule that verbs stay Spanish while American technical nouns stay English, which grades *instruction-following in Spanish*, not vocabulary), semantic (2.0 — Spanish intent to the correct tool name, graded strictly: naming the right tool **and no other**), and charset (1.0 — do the diacritics survive the round trip, or return as mojibake). Thresholds: $\geq 0.85 \rightarrow$ `FLUENT`, $\geq 0.45 \rightarrow$ `ASSIST`, otherwise `WEAK`.

Three properties of the probe matter more than its contents.

No model judges another model. Every grader is a pure function of the response string, so a verdict cannot drift between runs the way an LLM-as-judge verdict does — the same hazard §8.5 gates the evaluation judge against, avoided here by construction rather than by measurement.

The raw features are persisted, not just the verdict. Per-check scores and the model's own answers are stored alongside the tier, so thresholds can be retuned **without re-probing anything**, and a human can audit or override a verdict rather than merely observe it. A `PROBE_VERSION` counter invalidates verdicts whose features are no longer comparable; an older verdict is treated as absent rather than trusted.

A transport failure is not evidence. If more than one probe is unreachable, the run returns `INCONCLUSIVE` and **is never cached**. A dead connection says nothing about Spanish, and caching it would convert an outage into a permanent verdict.

L2 — passive verification. The same graders run free on real production responses whenever the user's request was Spanish: zero tokens, microseconds of CPU. Its authority is deliberately one-directional.

O1. *Observation may* ***demote*** *a standing verdict. It may* ***never*** *promote one — promotion requires a probe, because a run of easy answers is not evidence.*
O2. *Demotion requires a* sustained *failure rate (below 0.60) over a* real *sample (at least 12), within a bounded window (200, halved on overflow) — so neither does an old run of failures hold a model down forever, nor does a good model bank credit against a later collapse.*

The asymmetry between L1 and L2 also repairs a tension the earlier draft of this section left open. Probes run at temperature 0 with a fixed seed where the transport supports it, because a verdict that cannot be reproduced cannot be audited or retuned — but probing at temperature 0 systematically measures the model's **best** case, which biases the profile optimistic, and optimism is precisely the harmful direction. The resolution is not to abandon reproducibility; it is to let the two levels compensate. L1 is reproducible and optimistic, and is held to a deliberately high `FLUENT` threshold for that reason. L2 runs at production temperature on production traffic, and carries the demote authority. The one that can be trusted to be exact is not allowed to be generous; the one that sees reality is not allowed to be flattering.

Identity — the cache key.

A verdict is cached against an identity that changes if and only if the artifact changes: the **GGUF blob sha256 digest** for a local model, which moves when the weights or the tokenizer move and not otherwise; and the **exact model id** for a cloud model, including the `:cloud` suffix and any size tag.

Never the family name. An earlier revision collapsed `glm-5.2:cloud` and `glm-5.2` to a single key, so a local artifact and a cloud endpoint that merely share a family name would have shared one verdict — a measurement of one thing attributed to another. This is pinned by a test. It also subsumes part of the freshness problem: a local model that is re-quantized or re-tokenized gets a new identity automatically, and a cloud alias that is silently swapped behind a stable id is caught not by a version string but by L2's demotion authority, which does not care *why* the answers got worse.

Fail-open, and the direction of the asymmetry.

Every failure path resolves to `ASSIST` — the safe middle — and never to `FLUENT`. Nothing in the module raises into a caller and nothing blocks a request. The direction is not a default; it is an argument.

A wrong `ASSIST` costs some hint tokens and slightly noisier scoring on a request the model would have handled unaided, and it is corrected by the next probe. A wrong `FLUENT` silently withholds the help a weak model needed, produces *"no tool matched"* on a request the system could have routed, and reads to the operator as Tlamatini simply failing in Spanish — with no error, no log line and no way for the user to attribute the failure. The two errors are not symmetric in cost, in detectability, or in recoverability, so the fail-open direction is fixed by the one that is cheap and self-correcting rather than by the one that is silent and durable.

`UNKNOWN` is therefore not a fourth behaviour. It is `ASSIST` under a different name, and the name is kept only so that the operator-facing profile can distinguish *"measured as adequate"* from *"never measured"*.

What the profile emits.

Table 17: What the profile emits. Folding is unconditional; expansion is what the tier actually decides.

Tier	Stage 2 policy (fold · expand · boost)	Intended ladder start
`FLUENT`	fold only	R0
`ASSIST`	fold + expand	R1
`UNKNOWN`	fold + expand — *identical to* `ASSIST`, *deliberately*	R1
`WEAK`	fold + expand + boost	R1

Folding (N1) is free and always on: it is the identity on ASCII, so it costs nothing and is never gated. Expansion (N3) is what the tier actually decides. Embedding models are excluded from the ladder entirely — they never produce prose, `/api/generate` rejects them outright, and probing them is meaningless.

The module is **stdlib-only and never calls an LLM**: `run_probe` takes an `invoke(prompt) -> str` callable supplied by the caller. It is therefore importable from a pool subprocess, testable against a fake, and *structurally* incapable of stalling the application — which is the mechanical form of §6.11's rule that the profile is computed out of band, one global worker, never on the

request path, refusing to run while a generation is in flight.

Honest limits.

Four, stated plainly, because a measurement section that reported only its successes would be the thing this paper argues against.

The thresholds are unvalidated. 0.85, 0.45, 0.60 and the sample floor of 12 encode the failure modes described above — they do **not** encode a measured correlation with MMLU-es or any other external Spanish benchmark. No such validation has been performed. They are defensible choices, not calibrated ones, and §8's instrumentation is where they would first be tested against outcomes.

Four tasks is a smoke test, not a benchmark. The battery detects gross deficiency across four axes. It does not measure long-form, hard, or domain-specific Spanish, and a model that passes it has demonstrated only that it does not fail in the four ways that break this system first.

The probe conflates instruction-following with language ability. A model that ignores *"responde en español"* fails the language check, and it is arguable that what has been measured there is compliance rather than competence. For this system's purposes the conflation is tolerable — an operator system needs a model that *does what it is told in Spanish*, not one that could in principle — but it is a conflation, and a verdict of `WEAK` should not be read as a claim about the model's Spanish in general.

The gate is implemented and tested, and not yet firing. `agent/i18n/model_caps.py` and its 45 tests are complete, but every production call site of `normalize_request` — four of them, across `capability_registry.py`, `global_execution_planner.py` and `mcp_agent.py` — still invokes it with no model name, so today every request takes the legacy path and the tier is never consulted. Threading the model identity through the planner and the executor is a core change that remains pending. Relatedly, the module emits a **tier and a Stage-2 policy**, not a ladder start rung: the tier-to-rung column in Table 17 is *specified* here as the intended binding of §6.7's `profile.start_rung`, and is not yet code. Nothing in §6's guarantee depends on this — by Corollary 1 an unconsulted profile is simply a profile that always starts at the conservative rung — but the cost model of §6.11 is a projection until it does.

7.4 Three worked traces

Trace A — a Spanish-competent model.

> u = "Crea un archivo en `C:\Tlamatini\Temp\notas.txt` con el texto 'hola mundo' y luego muéstrame las últimas 5 líneas del log"

Trace B — a Spanish-blind model, and the provenance check earning its place.
Same utterance, a small English-centric local model.

Trace C — a destructive request with no path, which today fails silently.

> u = "Borra los archivos temporales de esa carpeta"

Trace C (Table 20) is the strongest argument for the architecture. The alternative behaviours are all worse: today's system tells the user no tool is needed; a naive Spanish port would let a

Table 18: Trace A — a Spanish-competent model.

Step	What happens
Freeze	Σ = { `C:\Tlamatini\Temp\notas.txt`, `hola mundo`, `5` }
Profile	competent → start at **R0**
R0 proposal	`chat_agent_file_creator(file_path='C:\Tlamatini\Temp\notas.txt', content='hola mundo')`, then a read of the log tail
Verify	V1 $\surd$ V2 $\surd$ V3 $\surd$ (both literals trace to Σ) V4 $\surd$ V5 $\surd$ V6 $\surd$ V7 $\surd$
Execute	file created at the exact path
Render	model already answered in Spanish; **File-Creator** appears verbatim in the Exec Report caption
Cost	**1 model call — identical to English**

Table 19: Trace B — a Spanish-blind model, and the provenance check earning its place.

Step	What happens
Freeze	Σ = { `C:\Tlamatini\Temp\notas.txt`, `hola mundo`, `5` }
Profile	unknown/weak → start at **R1**
R1 proposal	`chat_agent_file_creator(file_path='C:\Tlamatini\Temp\notes.txt', content='hello world')`
Verify	**V3 REJECTS.** `...\notes.txt` $\notin \Sigma \cup \mathcal{R} \cup \mathcal{D}$, and `hello world` $\notin \Sigma$. The model translated a *filename* and a *content literal*
—	*Without NEPANTLA this executes successfully and creates the wrong file with the wrong contents, reporting success.*
Escalate	**R2** — Spanish original + English gloss (literals masked and re-injected) + anchor table
R2 proposal	correct path, correct content
Verify	all checks pass
Execute	correct file created
Render	model cannot write Spanish → **delegated verbalization**; Exec Report chrome from the Spanish catalog; **File-Creator** verbatim
Cost	**2 model calls; correct outcome; user reads Spanish**

Table 20: Trace C — a destructive request with no path, which today fails silently.

Step	What happens
Freeze	$\Sigma = \emptyset$ — **there is no path literal**; "esa carpeta" is anaphoric
Neutralize	N1/N2/N3 lift the request into canonical keys → planner expects a **destructive file operation**
—	*Today: the scorer returns zero positives, the planner emits "Selected tools/agents: none", and the model is told no tool stage is needed — for a delete.*
R1 proposal	`chat_agent_deleter(target_path='C:\Tlamatini\Temp')` — a path the model **invented**
Verify	**V3 REJECTS.** The path has no provenance: not in Σ, not in $\mathcal{H}$, not a declared default
Escalate	R2, R3 — each proposal that fabricates a target is rejected for the same reason
R4	**Honest stop, in Spanish:** *"No puedo ejecutar el borrado: no me diste la ruta de la carpeta y no hay ninguna en el historial de esta conversación. Dime la ruta exacta y lo hago."*
Execute	**nothing**

model guess a directory and delete it. NEPANTLA refuses, explains, and asks — because a literal with no provenance is treated as an emergency rather than as a detail.

Note also that if the folder *had* been established in a prior turn, it would be in $\mathcal{H}$, provenance would pass, and V5 would route the action through the Ask-Execs permission prompt — rendered in Spanish, naming **Deleter** in English.

7.5 The guarantee, restated for a hostile model

Take the worst admissible case: a model with no Spanish comprehension, no Spanish generation, and a tendency to normalize literals. NEPANTLA's behaviour is then:

- Execution: escalates to R3, which *is* the English pipeline, with byte-exact literals. $\theta \approx \theta_{EN}$**.**
- Arguments: protected absolutely by V3. **Zero corruption.**
- Protocol: protected by V6; operator route pinned English for that model. **Zero silent routing damage.**
- Presentation: Spanish via the renderer, or Spanish-by-template as a floor. **The user never sees English.**
- Cost: 2–4 model calls instead of 1.

The user pays latency. They do not pay correctness, and they do not pay language.

8 Falsifying the Guarantee

An architectural argument that has never been measured is a hypothesis in a proof's clothing. §6.8 leaves exactly one condition to be established empirically ($\theta_{R_{m-1}} \geq \theta_{EN}$) and one quantity to be bounded (FA on the undecidable residue). This section specifies how.

8.1 Non-inferiority, pre-registered

Parity is a one-sided claim about a margin, not a failure to reject equality:

$$H_0 : \Delta \leq -\delta \qquad \text{vs} \qquad H_1 : \Delta > -\delta, \qquad \Delta = \theta_{ES} - \theta_{EN} \tag{3}$$

at one-sided $\alpha = 0.05$, claimed only if the lower bound of the one-sided 95% interval exceeds $-\delta$. The naive alternative — report $p > 0.05$ and call it parity — is *perversely incentivized*, because a smaller and noisier study is more likely to produce it. Under this framing a weak study yields a wide interval and correctly **fails**.

δ is the minimum of three independently justified bounds: operational relevance (at $\theta_{EN} = 0.85$, five points raises the failure rate by a third), fraction-of-effect retention against a tools-unbound baseline, and the harness's own reproducibility floor σ_{run} — you cannot certify a margin finer than your own noise. Primary $\delta = 0.05$ absolute; sensitivity reported at $\{0.03, 0.05, 0.10\}$.

8.2 Assay sensitivity — the part usually omitted

A non-inferiority design in which the harness *cannot detect a real deficit* is unfalsifiable. Three deliberately degraded positive controls are therefore mandatory, and **the parity conclusion is void unless the pipeline rejects non-inferiority for all three**:

1. a Spanish arm with the tool-usage rules stripped from the system spine;
2. an arm in which 10% of Spanish prompts have their literal arguments corrupted *after* freezing (so V3 cannot rescue them);
3. an arm running a deliberately weaker model.

8.3 Paired analysis and power

The same task runs in both languages, so outcomes are paired. Note the category error to avoid: **McNemar's test measures equality and cannot test a margin** — it is reported as a companion, never as the decision. The margin test is Tango's efficient-score interval for the paired difference.

Sample size is driven by the discordance rate p_d, not the base rate: $n = 2473\, p_d$ at $\delta = 0.05$.

Table 21: Required paired sample size as a function of the correlation between arms.

ρ	p_d	n_{pairs}	expected discordant
0.00	0.255	631	161
0.60	0.102	253	26
0.70	0.077	190	15
0.80	0.051	127	6

The $\rho = 0$ row reproduces the unpaired 631-per-arm answer, the correct sanity check. Plan: a 50-item pilot to estimate p_d and σ_{run}, then **250 paired items per model**, with a floor of ~25 expected discordant pairs.

8.4 Endpoints

The primary endpoint is **programmatic** — an executable success predicate evaluated against the machine in a fresh sandbox. **The disk does not speak Spanish or English**, which is exactly why this endpoint is immune to language-similarity confounds. Incomplete, crashed or timed-out runs count as failures; dropping them biases toward parity.

Table 22: Evaluation endpoints, whether an LLM judge is involved, and what each one is for.

Endpoint	Judge?	Purpose
Task Success Rate (primary)	No	The claim
Literal Drift Rate — any literal differs between the ES and EN runs of one item	No, **and gold-free**	Direct measurement of the C3 failure class
Tool-Selection Accuracy — exact multiset / sequence match	No	The C2 class
Argument Exactness — byte-identical after **NFC only**	No	Proposition 2 in practice
Ask-Execs Gating Parity — identical gated-tool set in both arms	No	*Safety*, not quality
Answer-Language Correctness — LPR/WPR on **masked** prose	No	The C1 class
Plan-Length Parity	No	Efficiency regression
Refusal justification, prose adequacy	Yes	Minority of the surface

NEPANTLA-specific instrumentation, which is what makes the theorem auditable rather than decorative:

- **Rung distribution** — the histogram of terminating rung per model. This *is* the cost model of §6.11, measured.
- **Verifier rejection histogram by reason** (V1…V7). A design that never rejects is not verifying; a design that rejects constantly has a broken extractor.
- **False-accept estimate** — human adjudication of a stratified sample of *accepted* proposals, giving the FA that Theorem 1's bound needs.
- **False-reject rate** — how often escalation was unnecessary. This is the pure latency tax.
- **R4 rate** — how often the system honestly stopped, and whether those stops were justified.

8.5 Instruments and hygiene

No new dependency. Cross-lingual similarity is computed through the already-present embedding endpoint; character-level metrics are ~50 lines of standard library; the language identifier is a closed-set stopword-plus-trigram classifier. Two traps are avoided explicitly: BLEU across the language pair is *inadmissible* (the two answers are supposed to differ in surface form, so overlap carries no parity information), and the default embedding model is English-only, so an ES↔EN cosine computed with it is meaningless noise that *looks plausible* — a multilingual encoder is required for evaluation.

The judge is measured before it is trusted. Language bias is the decisive one here: judges score non-English higher and agree with humans less (Hada et al., 2024), so a biased judge would *manufacture* parity. Protocol: pointwise rubric-anchored discrete scores, blinded language labels, an odd panel from different model families, no judge scoring its own family — and a gate on the *differential* bias $b_{ES} - b_{EN}$ with a bootstrap interval, measured against a native Mexican-Spanish reviewer who is also an operator. Report Cohen's κ **and** Gwet's AC1 per language, because at an 85% base rate skewed marginals collapse κ even at high raw agreement.

Determinism is measured, not configured. Temperature 0 does not make a model deterministic on shared infrastructure — the forward pass is not batch-invariant, so output depends on the batch a request lands in (He et al., 2025). Set seeds where accepted, *document which arms could not be seeded*, run $k = 5$ repeats, report variance components, and separately prove the scoring harness is byte-deterministic with a record-replay fixture. Pin the state hazards before each batch: assert identical enabled tool/agent sets across arms (the agent table is rebuilt on every server start), freeze the clock, reset the sandbox between items, re-assert the toolbar flags at every send, and read all subprocess output as UTF-8 with replacement — a legacy-codepage read corrupts Spanish *before* scoring and manufactures failures that are pure harness artifacts.

8.6 Corpus

250 paired items over seven risk-weighted strata (pure QA 40 as a *negative control*; single-tool 50; multi-tool $\geq$3 calls 45; literal-heavy 40; hardware in dry-run 25; messaging with sandbox recipients only 20; flow authoring 30).

The source of truth is a **language-neutral task specification** — intent, required tools, required literals, executable predicate — of which the English and Spanish prompts are two *renderings* with byte-identical literals enforced by automated diff. That construction is what makes Argument Exactness and Literal Drift Rate valid measurements rather than translation scores.

Translation follows a professional workflow (translate → independent revision → in-country review by a native Mexican-Spanish operator). Machine translation and back-translation QA are rejected: MT errors would be confounded with product errors, and MT systematically *simplifies* syntax, which would make the Spanish arm artificially easy. Roughly **half the items are authored Spanish-first**, because a set built entirely as translations of English inherits English discourse structure and systematically overestimates parity. Around 15% of items carry deliberate locale traps: decimal comma, es-MX date order, accented and ñ filenames, 24-hour times, the *billón* false friend, mixed encodings, tú/usted register variation, and realistic code-switching (*"hazle un git push al repo"*).

One operational warning: the messaging agents are deliberately ungated by the permission broker, so a messaging stratum run without sandbox recipients will contact real people.

9 Threats to Validity

Construct. The primary endpoint measures machine outcomes — right for an operator, blind to qualities a user cares about such as tone and clarity. The judge covers that residue only after passing the language-bias gate; if the gate fails, that part of the construct is unmeasured and must be reported as such rather than replaced by a similarity proxy.

The monotonicity caveat. Theorem 1 guarantees the *ladder* property unconditionally but assumes the terminal rung is at least as good as the baseline. Additional context is not provably monotone for a language model. This is stated as a condition, not smuggled in as a lemma, and §8 measures it directly.

The undecidable residue. V cannot adjudicate which of several schema-valid tools is wisest. NEPANTLA inherits the baseline there. If a future failure analysis shows that Spanish specifically degrades *that* choice, the remedy is a stronger planner, not a stronger verifier — and the ladder would need a new rung.

Extraction coverage. Proposition 2's guarantee is exactly as good as Σ. A literal the extractor fails to recognize is unprotected. This is why the extractor over-extracts by design, and why extraction recall is itself an evaluation target rather than an assumption.

External validity. The operator is in Mexico; most published Spanish benchmarks measure Peninsular Spanish, and localization is not translation. "Spanish" is not one language. The in-country review is a mitigation, not a solution.

Internal validity. Public multilingual suites carry contamination risk — MEGAVERSE explicitly warns that several evaluated models are likely contaminated with multilingual benchmarks (Ahuja et al., 2024) — which is one reason the primary endpoint is a private, specification-derived corpus. Translated benchmarks also confound translationese with capability: a 13-point HellaSwag EN→ES gap sits beside a 3-point MMLU gap for the same model in the same evaluation (Thellmann et al., 2024), a pattern far better explained by translation artifacts than by a commonsense deficit.

Scope of the §5 measurement. The 0/12 result characterizes the *scorer*. Because Multi-Turn binds the full tool surface when it fits, a Spanish prompt may still succeed on the model's own choice. The deterministic damage is confined to the planner's graph, the system-prompt summary and the bind ranking under overflow — which is why the claim is *"the correct tool is not reliably selected"* and not *"Spanish does not work at all"*.

Security transfer. Agent-security attack success is measured at 60.2% in Spanish against 53.4% in English (Hofman et al., 2026). English-tuned injection defenses under-protect Spanish users, which is why Ask-Execs Gating Parity is its own hypothesis in §8 rather than a footnote.

Bibliographic integrity. Every reference below was independently re-fetched to confirm title, authors, year and identifier, and every numeric claim was checked against its source table. One reference was removed during that pass for misattribution — a real paper cited for figures it does not contain. We record this because a paper claiming rigor should show how its own errors were caught.

10 Conclusion

The problem looked like a translation problem and was not. For Spanish the model-side deficit is 1–4 points on reasoning and approximately zero on tool selection; the real obstacle was that the system's own deterministic control plane was monolingual at the tokenizer level, and that no amount of model quality could compensate for a user-chosen model whose Spanish nobody had measured.

NEPANTLA resolves both by refusing to depend on the answer to the unanswerable question. It **freezes** the user's literals before anything can touch them, **neutralizes** the deterministic core so that intent — not the language intent arrived in — drives routing, **proposes and verifies before it executes**, and **escalates** through a ladder whose last rung is the English pipeline itself. Because rejected proposals have no side effects, the ladder cannot end below its own terminal rung; because the verifier's checks are decidable on exactly the failure classes cross-lingual operation introduces, the false-accept rate on those classes is zero; and because presentation is causally downstream of the verified action, the operator reads Spanish without any of that reaching the machine.

Three consequences deserve to be carried away from this paper.

Spanish competence became an optimization rather than a dependency. A model with no Spanish at all still produces correct execution and a Spanish answer; it merely costs one or two extra calls. That is the direct answer to "the user can pick any model".

The naming rule is a theorem, not a preference. `Emailer` stays `Emailer`, `Asker` stays `Asker`, `Apirer` stays `Apirer`, every tool name, flag, configuration key, sentinel and identifier stays exactly as it is — because Proposition 1's independence, and therefore the entire guarantee, holds only while the action vocabulary is the same set of symbols in every locale. The Spanish build translates what the operator reads and nothing the machine reads. That line is drawn in §2.3 and enforced by Corollary 2.

And the Spanish build may end up better than today's English one. Three of the required repairs — boundary-aware phrase matching, the failure-classifier polarity, and locale-invariant child processes — fix defects that damage the English path right now. Argument provenance protects an English user's paths exactly as well as a Spanish user's. The honest stop at R4 prevents a fabricated deletion in either language.

Localization, done this way, is not a translation project attached to the side of a product.

*It is a **correctness project that happened to be discovered by asking the system to speak Spanish** — and the discipline it forces, verify before you act and never let a decision rest on a literal nobody said, is worth having regardless of the language anyone is speaking.*

A What Is Queryable About a Model's Language Support: An Empirical Survey

§7.3 specifies a capability profile and asserts, without much ceremony, that it must be *measured*. That assertion is not free. It commits the system to spending model calls, maintaining a cache, versioning a battery of graded tasks, and defending a set of thresholds — all to answer a question that, on the face of it, an API ought to answer for nothing. This appendix establishes that no API answers it, that no model file answers it reliably, and that the cheapest plausible proxy does not survive contact with the data. The measurement in §7.3 is therefore a **forced move**, not a design preference.

The result is negative, and the negative result is the contribution. What follows is the survey we would have wanted to read before building the profile, so that the next system does not have to rediscover it.

A.1 Motivation — the design that should have worked

The naive architecture is a lookup, and it is the obvious first choice for good reasons. Every other capability NEPANTLA depends on is already declared: whether a model accepts images, whether it supports tool calling, how large its context window is, whether it emits structured output. These are read from a registry at startup, cost nothing, never drift, and require no probe. Language support is *the same kind of property* — a static fact about a model artifact, fixed at training time, of obvious interest to anyone integrating it. If context length is queryable, supported languages should be queryable.

Under that design §7.3 collapses to three lines: read the model's declared language list, test for `es`, and route. There is no battery, no cache, no threshold, no `PROBE_VERSION`, no risk of a stale verdict, no tokens spent, and no possibility that the profile itself becomes a source of error. Corollary 1 would still hold, but it would be defending against nothing.

The design was rejected because it cannot be implemented. Not "is currently awkward" — **cannot be implemented against any interface that exists.** Establishing that took a survey of five provider APIs, a parse of fifteen model files, a live interrogation of a thirteen-model local host, and a measurement of the leading proxy metric on seven of those models. Each step is reported below, with the queries, so that a reader can rerun it and either confirm the finding or refute it on their own hardware.

A second motivation deserves stating. The alternative to a lookup is not a probe — it is a **hardcoded table of model names**, which is what most systems ship. Such a table is a maintenance liability that silently rots: it cannot know about a model released after the table was written, it cannot distinguish two artifacts that share a family name, and it fails in the unrecoverable direction, because a model absent from the table is indistinguishable from a model known to be bad. Every argument below against a lookup is *also* an argument against the table. If the fact cannot be read and cannot be listed, it must be measured.

A.2 Method

Provider schemas. For each of five providers, the model-enumeration endpoint was queried against primary documentation and, where an endpoint was reachable, against a live response. The recorded artifact is the complete field list the endpoint returns — not a summary — because the claim being tested is an absence, and an absence is only credible against an exhaustive field list.

Ollama `/api/show`, twice. The local host at `127.0.0.1:11434` served thirteen models at the time of measurement. `POST /api/show` was issued against two deliberately contrasting cases: a **cloud-served** model (`gemma4:cloud`), where Ollama brokers a remote artifact it does not hold weights for, and a **locally-weighted** model (`qwen3-vl:8b`), where the GGUF blob is on disk and every metadata key is in principle readable. The two responses were compared key-by-key. This contrast is the load-bearing part of the method: a metadata channel that works only for local models is not a channel a system can route on, because the operator may bind either.

GGUF `general.languages`. Fifteen model files were parsed at the header level and the `general.languages` key extracted where present. Both the *presence* and the *contents* were recorded, because the two fail differently and the second failure is the more dangerous one.

Token accounting. Because `/api/show` returns `tokenizer.ggml.tokens` as `None` (§A.3.3), the vocabulary is not available for local tokenization, and fertility must be measured **behaviourally** — by submitting text and reading the prompt token count the server reports back. Three controls were applied:

1. **Same meaning, both languages.** English and Spanish renderings of identical content, so that the comparison is not confounded by the two texts saying different things.
2. **Baseline subtraction.** A fixed carrier is measured on its own and subtracted, so that the chat-template and system-prefix overhead — which is per-model and has nothing to do with the language under test — does not enter the ratio.
3. **A diacritics-only control.** The *same Spanish words* were submitted twice, once correctly accented and once with the diacritics stripped. Because the word sequence is identical, any difference in token count isolates the tokenizer's handling of accented characters from Spanish's ordinary verbosity relative to English. This control is what separates "Spanish is longer" from "Spanish is badly segmented", and the two are routinely conflated.

The reported statistic is Spanish-per-character efficiency: chars/token in Spanish divided by chars/token in English, baseline-subtracted. A value of 1.0 means the tokenizer is as efficient on Spanish as on English; lower values mean Spanish costs more tokens per unit of meaning.

What was not done. No external Spanish benchmark was run, and no correlation between any of these signals and downstream Spanish task accuracy was established on this hardware. That limit is load-bearing for §A.5 and is not softened here.

Table 23: The complete field set returned by the model-enumeration endpoint of five providers. The claim under test is an absence, so each field list is exhaustive rather than summarized.

Provider	Endpoint	Complete field set returned	Language field
Anthropic	`GET /v1/models`	`id`, `display_name`, `created_at`, `type`, `max_input_tokens`, `max_tokens`, `capabilities{batch, citations, code_execution, context_management, effort, image_input, pdf_input, structured_outputs, thinking}`	**none**
OpenAI	`GET /v1/models`	`id`, `object`, `created`, `owned_by`	**none**
Google	`models.list`	`name`, `version`, `displayName`, `description`, `inputTokenLimit`, `outputTokenLimit`, `supportedGenerationMethods`	**none**
Ollama	`/api/show` → `capabilities[]`	exactly 8 values: `completion`, `tools`, `insert`, `vision`, `embedding`, `thinking`, `image`, `audio`	**none**, and no language member in the enum
OpenRouter	`GET /api/v1/models`	the richest cross-provider schema in existence — `architecture`, `tokenizer`, `pricing`, `modalities`	**none**, confirmed by direct inspection

A.3 Results

A.3.1 No provider exposes supported languages

The OpenAI row is worth pausing on: four fields, one of which is a literal type tag and one a timestamp. That is the *complete* schema. The OpenRouter row is the decisive one, because OpenRouter's entire commercial function is to normalize heterogeneous model metadata across providers into one comparable surface — it publishes tokenizer family, modality lists and per-token pricing. If any aggregator had a language field to expose, it would be this one. It does not.

The Anthropic row demonstrates that this is not an artifact of immature APIs. Anthropic's `capabilities` object is *granular* — it distinguishes PDF input from image input, and structured outputs from tool use. A schema with that resolution has clearly been designed rather than defaulted. Language is simply not in the design.

Finding 1. *Across five providers, including the aggregator whose business is metadata normalization, the number of endpoints exposing supported languages is* ***zero****. No lookup can replace a hardcoded model-name table, because there is nothing to look up.*

A.3.2 The one metadata field that exists is unusable

`general.languages` is a real GGUF metadata key, and it is the single most tempting signal in this entire survey, because it is precisely the field the naive design wants.

Both failure modes are present simultaneously, and they compound. Nearly half the files simply omit the key, and the omissions are not random — they are concentrated in models that unambiguously handle Spanish, so *absence correlates with nothing.* Where the key is present it

Table 24: Measured behaviour of the GGUF `general.languages` key across a fifteen-file corpus. Both failure modes — omission and misdeclaration — are present simultaneously.

Property	Measurement
Present	**8 of 15** model files parsed
Absent from	`qwen3`, `gemma3`, `gemma-2-9b`, `Mistral-7B-v0.3`, `Llama-2` — all strongly multilingual
Wrong where present	`Qwen2.5-7B-Instruct` declares exactly `["en"]`; `Phi-4` declares `["en"]`

can still be flatly wrong: Qwen2.5-7B-Instruct declaring a single language is not a marginal call, it is a declaration contradicted by the model's own behaviour.

The reading path makes it worse. Ollama's `server/routes.go` blanks any metadata array longer than five entries unless the request sets `verbose: true`. The consequence is not degradation but **inversion**:

Table 25: The truncation inversion. Ollama's five-entry blanking rule preferentially destroys long language lists, and long lists are exactly the multilingual ones.

Model	True `general.languages`	As read through Ollama, default	Naive conclusion
Llama 3.1	8 entries, **containing `es`**	`[]` — blanked for exceeding 5	"declares no languages"
Phi-4	`["en"]` — misleading	`["en"]` — survives, 1 entry	"declares English only"

A reader who takes the field at face value concludes *"no declared languages"* for the model that genuinely speaks Spanish and *"English only"* for the model that does not, because the truncation rule preferentially destroys long lists and long lists are exactly the multilingual ones. The signal is not merely noisy; **its noise is anti-correlated with the truth.**

Finding 2. *`general.languages` is absent from half the corpus, wrong where present, and read through a truncation rule that systematically deletes multilingual declarations while preserving monolingual ones. It cannot be used, and using it naively is worse than ignoring it.*

A.3.3 What `/api/show` actually returns — cloud versus local

Both queries were issued against the same host, seconds apart.

Three things follow. First, the cloud path returns **four keys and no tokenizer data at all** — a system that routes on model metadata is blind for every cloud-served model, which on Angela's host is the majority and includes the primary configured model. Second, even the local path withholds the vocabulary: `tokenizer.ggml.tokens` is present as a key with a `None` value, so vocabulary size is not derivable without `verbose: true`, and any fertility computation must therefore be behavioural rather than local. Third — and this is the honest positive finding of the survey — **`capabilities[]` is real, dynamic, and returned identically for cloud and local artifacts.** The channel works. It carries `vision`, `tools`, `thinking`. It is simply, and deliberately, silent about language.

Table 26: `/api/show` on the same host, seconds apart: a cloud-served model against a locally-weighted one. The metadata channel functions in both cases and carries no language fact in either.

	`gemma4:cloud` (cloud-served)	`qwen3-vl:8b` (local weights)
`model_info` keys	**4**	**39**
The keys	`gemma4.context_length`, `gemma4.embedding_length`, `general.architecture`, `general.parameter_count`	architecture, tokenizer and dimension metadata
`tokenizer.ggml.tokens`	absent	present but `None` — payload omitted
`general.languages`	**absent**	**absent**
`general.tags`	**absent**	**absent**
`general.datasets`	**absent**	**absent**
`capabilities[]`	`["completion","thinking", "tools","vision"]`	`["completion","thinking", "tools","vision"]`

Finding 3. *The metadata channel is not broken. It is complete, functioning, and does not contain the fact. This is why the absence is structural rather than incidental: language support is not a field anyone forgot to add.*

A.3.4 Tokenizer fertility, measured and rejected

Fertility is the best available proxy: it is cheap, it is objective, it requires no judgement, and there is real literature relating it to accuracy. It was measured on seven models.

Table 27: Behavioural fertility on seven models. Every measurement falls inside a nine-point band, and the band's most English-biased entry is the frontier model with the best Spanish.

Model	Spanish-per-character efficiency (ES ÷ EN, baseline-subtracted)
`glm-5.2:cloud`	**0.76**
`qwen3.5:cloud`	0.83
`gemma4:cloud`	0.83
`gpt-oss:120b`	0.83
`qwen3-vl:8b`	0.74
`qwen3-vl:4b`	0.74
`Orpheus-3b`	0.74

Every model lies in **0.74–0.83** — a nine-point band, on a scale where the theoretical range is unbounded below and 1.0 above. There is no threshold that partitions this band usefully. Any cut placed inside it separates models that differ by three hundredths, and the specific damage is immediate and disqualifying: **any threshold that separates this band mislabels `glm-5.2:cloud` — a 756B frontier model with excellent Spanish, and Angela's primary configured model — as the most English-biased artifact measured.** The metric orders the corpus with its best Spanish model last.

The diacritics-only control produced one incidental observation, recorded here for completeness and relied upon for nothing. The "accent surcharge" — the token-count difference between correctly accented Spanish and the same words with diacritics stripped — came out **negative for every real multilingual model**: correctly accented Spanish tokenizes *more cheaply*, because `cuánto` is a single vocabulary entry while the misspelled `cuanto` splits. It was positive only for `Orpheus-3b`, a 3B English fine-tune, where the accented forms fall back to bytes. That is suggestive of a byte-fallback detector, and it is one data point against six. It is not a signal this system routes on.

A.4 Analysis — why each candidate fails, and what the literature predicts

The provider APIs fail by design, not by omission. Language support is not a discrete capability of the kind an API can honestly advertise. `vision` is binary: the model either accepts image tokens or it does not. `tools` is binary: the schema is either supported or rejected. Spanish is a **continuum crossed with a task**, and it is not even one continuum — a model can write fluent Spanish prose and still mangle a `chat_agent_stm32er` argument list, which is exactly the C1/C3 split of §7.1. A provider asked to publish a language list would have to pick a threshold and a task, and would be wrong for every consumer whose threshold or task differed. Publishing nothing is the defensible engineering choice. It is also, for our purposes, fatal.

`general.languages` fails because it is a training-time annotation, not a measurement. It records what the model card's author chose to write down, propagated through whatever conversion tool produced the GGUF. Nothing validates it, nothing updates it, and no downstream consumer's complaint reaches it. Qwen2.5-7B-Instruct declaring `["en"]` is not a bug in a system; it is a field nobody had a reason to be careful about. Metadata that is never read is never corrected.

Fertility fails for a reason the literature already documents: it discriminates across wide ranges and not within narrow ones.

- In a wide range it genuinely works. Across 10 LLMs and 16 African languages, regression slopes of -0.08 to -0.18 accuracy per additional token per word explain 20–50% of variance (arXiv:2509.05486 — AUTHORS NOT VERIFIED, 2025). That is a real effect, and it is why fertility is a reasonable first hypothesis.
- In a narrow range it does not. A Ukrainian zero-shot study reports $\rho = -0.43$ at $p = 0.34$ — not significant (arXiv:2605.14890 — AUTHORS NOT VERIFIED, 2026). The point estimate looks like a relationship; the interval says it is indistinguishable from noise.
- **Spanish is decisively the narrow case.** Across 24 European languages and six tokenizers, English averages 1.23 tokens per word and Spanish 1.46 (arXiv:2605.24718 — AUTHORS NOT VERIFIED, 2026) — an 8–29% band. Spanish is a Latin-script, low-inflection, high-resource language that every major tokenizer has seen abundantly. It sits at the easy end of the very distribution in which fertility is known to lose its discriminating power. Our own 0.74–0.83 measurement reproduces that band on independent hardware.

Two further objections are structural rather than statistical, and either alone would be sufficient.

The family ceiling. Every model in a family shares one tokenizer and therefore exactly one fertility number, while differing enormously in Spanish output quality — a 4B and a 235B sibling are indistinguishable to this metric by construction. `qwen3-vl:8b` and `qwen3-vl:4b` both measure 0.74 in our table, which is not a coincidence but a demonstration. A metric that cannot discriminate *within* a family cannot support per-model routing, because per-model routing is largely a within-family problem.

The counterexamples run backwards. `Phi-4` carries a 100,352-entry vocabulary and declares itself English-only. `Mistral-7B-v0.3` carries 32,768 entries and handles Spanish. Vocabulary size — the crudest fertility proxy — orders these two exactly the wrong way round.

Finding 4. *Fertility measures* ***cost****, and measures it well. It is the correct input to context budgeting and to per-token pricing, and NEPANTLA uses it for nothing else. It does not gate language routing, because within the range Spanish occupies it does not carry the signal, and because it is constant across a family whose members differ.*

That leaves exactly one route, and §7.3 takes it: **run four short graded tasks and read the answers.** The ladder in `agent/i18n/model_caps.py` is the direct consequence of this appendix — L0 is a name-shaped prior that can never demote, L1 is the probe and the authority, L2 is passive verification that can demote but never promote. The reason every grader is a pure Python function rather than a model judging another model is the same reason this survey exists: a verdict that depends on an unmeasured capability is not evidence, it is a restatement of the question.

A.5 Threats to validity

The English figures are degenerate, and this weakens §A.3.4 substantially. Characters-per-token for English came out **identical — 5.77 — across all seven models**, and Spanish took **only two distinct values**. For tokenizers from genuinely different families (GLM, Qwen, Gemma, GPT-OSS, and a Llama-derived 3B fine-tune) that is implausible on its face. Different byte-pair merge tables trained on different corpora do not segment the same English paragraph into the same number of tokens. The most likely explanation is that Ollama does not apply the true per-model tokenizer for `:cloud` models, and possibly applies a shared or approximate accounting path more broadly — which would mean the measurement is partly of Ollama's accounting rather than of the models' tokenizers.

We record two consequences rather than reconciling them. First, the ratios reported in §A.3.4 are baseline-subtracted and are therefore not a naive quotient of the two raw figures, which is why three distinct ratios can coexist with two distinct Spanish values; but that arithmetic does not rescue the underlying raw numbers, which remain degenerate. Second, and more important, **this cuts in the same direction as the conclusion.** If the fertility figures are partly an artifact of the serving layer, then fertility is *even less* usable as a routing signal than the narrow band alone suggests — a system routing on it would be routing on a property of Ollama. Had the discrepancy pointed the other way, toward a signal we were discarding, it would have obliged us to re-measure before rejecting. It does not, and we state it plainly rather than omitting a result that makes our own measurement look worse.

Single-host, single-day. All live measurements come from one machine on 2026-07-28 with thirteen models. The provider-schema results are from primary documentation and generalize; the `/api/show` and fertility results are one host's behaviour at one version of one server, and a different Ollama build could return a different key set.

Fifteen files is a small GGUF corpus and was not sampled randomly — it is what was on disk. The specific proportion (8 of 15) should not be read as a population estimate. The *pattern* — omitted in multilingual models, wrong when present — is the finding, and it does not depend on the proportion.

Absence of a field is not absence of the fact. A provider could expose language support through documentation, a model card, or a future endpoint. The claim here is narrowly that it is not *programmatically queryable through the model-enumeration endpoint*, which is the only channel a runtime router can use.

The thresholds this appendix motivates are themselves unvalidated. Rejecting fertility establishes that a probe is necessary; it does not establish that our probe is correct. The cut points in `model_caps.py` (0.85 fluent, 0.45 assist, 0.60 observed-failure demotion, minimum sample 12) encode the failure modes catalogued in §7.1, **not a measured correlation with MMLU-es or any other external Spanish benchmark**. A four-task battery is a smoke test, not a benchmark: it does not measure long, hard, or domain-specific Spanish, and it conflates instruction-following with language ability, since a model that ignores *"responde en español"* fails the language check for a reason that is arguably not linguistic. This is why the raw per-check features and the model's own answers are persisted — so thresholds can be retuned against a real benchmark without re-probing, and so a human can audit and override a verdict.

And the gate is not yet firing. The module is implemented and passing 45 tests, but the three existing callers still invoke `normalize_request(text)` with no model name, so every request currently takes the legacy path. Threading the model name through the planner and the executor is a core change still pending. Nothing in this appendix should be read as reporting production behaviour.

A.6 Conclusion

The question *"does this model speak Spanish?"* has no answer in any interface that exists. Five providers — including the aggregator whose entire product is metadata normalization — expose zero language fields. The one metadata key that nominally carries the answer is missing from half of a real corpus, wrong in the half where it is present, and read through a truncation rule that deletes multilingual declarations while preserving monolingual ones, inverting the evidence. The leading cheap proxy measures cost rather than competence, sits inside a nine-point band on Spanish, is constant across a model family by construction, orders its own counterexamples backwards, and on our hardware is partly confounded with the serving layer.

That is the whole of it, and it is a negative result: **the fact is not discoverable, it is only observable.** Anything a system claims to know about a backend model's Spanish must therefore be something the system went and found out — by sending it Spanish and grading what comes back, deterministically, with the raw features kept so the thresholds can be argued with later.

This is not a satisfying answer. A lookup would have been three lines and no tokens. But the alternative to measurement is not a lookup; it is a hardcoded table of model names that ages badly, cannot see a model released after it was written, and fails silently in the direction that hurts — because a model absent from the table looks exactly like a model known to be bad. **Every argument in this appendix against querying the fact is equally an argument against listing it.** Corollary 1 is what makes the resulting probe safe to be wrong about, and §7.3's fail-open-to-ASSIST asymmetry is what makes it safe to be wrong about *in one direction only*. The measurement is the price of not guessing.

A.7 Reproduction recipe

Every result above is reproducible in under an hour with an API key per provider and a local Ollama. Model identifiers below are the ones measured; substitute your own.

1 — Provider schemas. Capture the complete field set, not a summary; the claim is an absence.

```
curl -s https://api.anthropic.com/v1/models \
  -H "x-api-key: $ANTHROPIC_API_KEY" -H "anthropic-version: 2023-06-01" \
  | python -c "import json,sys; d=json.load(sys.stdin)['data'][0]; print(sorted(d))"

curl -s https://api.openai.com/v1/models \
  -H "Authorization: Bearer $OPENAI_API_KEY" \
  | python -c "import json,sys; print(sorted(json.load(sys.stdin)['data'][0]))"

curl -s "https://generativelanguage.googleapis.com/v1beta/models?key=$GOOGLE_API_KEY" \
  | python -c "import json,sys; print(sorted(json.load(sys.stdin)['models'][0]))"

curl -s https://openrouter.ai/api/v1/models \
  | python -c "import json,sys; print(sorted(json.load(sys.stdin)['data'][0]))"
```

Listing 6: Provider model-enumeration schemas: capture the complete field list from each endpoint.

Grep each field list for `lang`. Expect no hits. The OpenRouter call needs no key and is the fastest single check in this appendix.

2 — Ollama `/api/show`, cloud versus local. Run both; the contrast is the point.

```
for M in gemma4:cloud qwen3-vl:8b; do
  curl -s http://127.0.0.1:11434/api/show -d "{\"model\":\"$M\"}" \
  | python -c "
import json,sys
d=json.load(sys.stdin); mi=d.get('model_info',{})
print('$M', 'model_info keys:', len(mi))
print('  capabilities:', d.get('capabilities'))
print('  general.languages:', mi.get('general.languages','ABSENT'))
print('  tokenizer.ggml.tokens:', type(mi.get('tokenizer.ggml.tokens')).__name__)"
done
```

Listing 7: The cloud-versus-local `/api/show` contrast.

Expect 4 keys for the cloud model and 39 for the local one; `general.languages` absent from both; `tokenizer.ggml.tokens` of type `NoneType` on the local model; identical `capabilities[]` on both.

3 — The verbose-blanking inversion. Same model, two calls.

```
curl -s http://127.0.0.1:11434/api/show -d '{"model":"llama3.1"}' \
  | python -c "import json,sys; print('default:', json.load(sys.stdin)['model_info'].get('
    general.languages'))"
curl -s http://127.0.0.1:11434/api/show -d '{"model":"llama3.1","verbose":true}' \
  | python -c "import json,sys; print('verbose:', json.load(sys.stdin)['model_info'].get('
    general.languages'))"
```

Listing 8: The verbose-blanking inversion: an 8-entry list containing `es` reads as empty by default.

The default call returns `[]` for an 8-entry list containing `es`; the verbose call returns the list. Repeat against a model whose declaration has ≤ 5 entries (Phi-4) and observe that the misleading `["en"]` survives both calls. The truncation rule is in `server/routes.go`.

4 — GGUF `general.languages` across a corpus. Parse the header KV block of every `.gguf` on disk and record presence *and* contents separately. The two failure modes are independent: absence in `qwen3` / `gemma3` / `gemma-2-9b` / `Mistral-7B-v0.3` / `Llama-2`, and a wrong `["en"]` in `Qwen2.5-7B-Instruct` and `Phi-4`.

5 — Behavioural fertility. Because `tokenizer.ggml.tokens` is `None`, do not tokenize locally — read the prompt-token count the server reports.

```
count () {  # $1 = model, $2 = text  -> prompt tokens
  curl -s http://127.0.0.1:11434/api/generate \
    -d "$(python -c "import json,sys;print(json.dumps({'model':sys.argv[1],'prompt':sys.argv
    [2],'stream':False,'options':{'num_predict':0}}))" "$1" "$2")" \
  | python -c "import json,sys; print(json.load(sys.stdin)['prompt_eval_count'])"
}
```

Listing 9: Behavioural fertility: read the server's reported prompt-token count rather than tokenizing locally.

Then, per model: measure an empty carrier and subtract it from every reading; measure the English and Spanish renderings of the *same content*; report chars/token for each and their ratio. Finally run the diacritics-only control — the identical Spanish word sequence with and without accents — which is the only measurement that separates accent handling from Spanish verbosity.

Record the raw chars/token figures, not only the ratios. That is how the degeneracy in §A.5 surfaced: identical 5.77 for English across seven unrelated tokenizers, and two distinct Spanish values. Reporting ratios alone would have hidden it.

Sources cited in this appendix by arXiv identifier only, pending independent verification of title and authorship: arXiv:2509.05486 — AUTHORS NOT VERIFIED (2025) — 10 LLMs $\times$ 16 African languages; regression of task accuracy on tokens-per-word, slopes -0.08 to -0.18, 20–50% of variance explained. arXiv:2605.14890 — AUTHORS NOT VERIFIED

(2026) — Ukrainian zero-shot evaluation; fertility–accuracy correlation $\rho = -0.43$, $p = 0.34$. arXiv:2605.24718 — AUTHORS NOT VERIFIED (2026) — 24 European languages across six tokenizers; English 1.23 and Spanish 1.46 tokens per word. *These complement, and are consistent with, the tokenization-cost references already carried in the main bibliography* (Petrov et al., 2023; Nayeem et al., 2025).

References

K. Ahuja et al. MEGA: Multilingual evaluation of generative AI. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4232–4267, 2023.

S. Ahuja et al. MEGAVERSE: Benchmarking LLMs across languages, modalities, models and tasks. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 2598–2637, 2024.

arXiv:2509.05486 — AUTHORS NOT VERIFIED. TITLE NOT VERIFIED — Angela to complete, 2025. arXiv:2509.05486. Cited in PAPER-v2.md by arXiv identifier only. Content as cited: 10 LLMs and 16 African languages; regression of task accuracy on tokens-per-word, slopes -0.08 to -0.18, 20–50% of variance explained. Title, authors and venue pending independent verification.

arXiv:2605.14890 — AUTHORS NOT VERIFIED. TITLE NOT VERIFIED — Angela to complete, 2026. arXiv:2605.14890. Cited in PAPER-v2.md by arXiv identifier only. Content as cited: Ukrainian zero-shot evaluation; fertility–accuracy correlation $\rho = -0.43$, $p = 0.34$ (not significant). Title, authors and venue pending independent verification.

arXiv:2605.24718 — AUTHORS NOT VERIFIED. TITLE NOT VERIFIED — Angela to complete, 2026. arXiv:2605.24718. Cited in PAPER-v2.md by arXiv identifier only. Content as cited: 24 European languages across six tokenizers; English 1.23 and Spanish 1.46 tokens per word. Title, authors and venue pending independent verification.

J. Etxaniz, G. Azkune, A. Soroa, O. Lopez de Lacalle, and M. Artetxe. Do multilingual language models think better in English? In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), Short Papers*, pages 550–564, 2024.

R. Hada et al. Are LLM-based evaluators the solution to scaling up multilingual evaluation? In *Findings of the Association for Computational Linguistics: EACL 2024*, pages 1051–1070, 2024.

H. He et al. Defeating nondeterminism in LLM inference. Thinking Machines Lab, September 2025. URL https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/.

O. Hofman et al. MAPS: A multilingual benchmark for agent performance and security. In *Findings of the Association for Computational Linguistics: EACL 2026*, 2026.

D. Kang, S. Hwang, D. Kim, H. Kim, and G. G. Lee. Why do multilingual reasoning gaps emerge in reasoning language models? In *Findings of the Association for Computational Linguistics: ACL 2026*, 2026.

C. Liu, W. Zhang, Y. Zhao, A. T. Luu, and L. Bing. Is translation all you need? In *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2025.

Z. Luo, T. P. Kutralingam, O. N. Okoani, W. Xu, H. Wei, and X. Hu. Lost in execution: On the multilingual robustness of tool calling in LLMs. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026.

M. T. Nayeem, S. Alqahtani, M. T. R. Laskar, T. Mohiuddin, and M. S. Bari. Beyond fertility: Analyzing STRR as a metric for multilingual tokenization evaluation. In *Advances in Neural Information Processing Systems (NeurIPS) Workshop*, 2025.

A. Petrov, E. La Malfa, P. Torr, and A. Bibi. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

T. Sales Almeida, J. G. Alves Santos, T. Laitz, and G. Kerche Bonás. Ticket-Bench: A kickoff for multilingual and regionalized agent evaluation, 2025.

K. Thellmann et al. Towards multilingual LLM evaluation for European languages, 2024.

W. Xuan et al. MMLU-ProX: A multilingual benchmark for advanced LLM evaluation, 2025.

Z. Zhang and Y. Zhu. Enhancing tool calling in LLMs with the International Tool Calling Dataset, 2026.